Machine Exercise 3

Carl Justin de Guia, 2011-06521

```
clear
clc
close all

params = get_my_params("2011-06521");
```

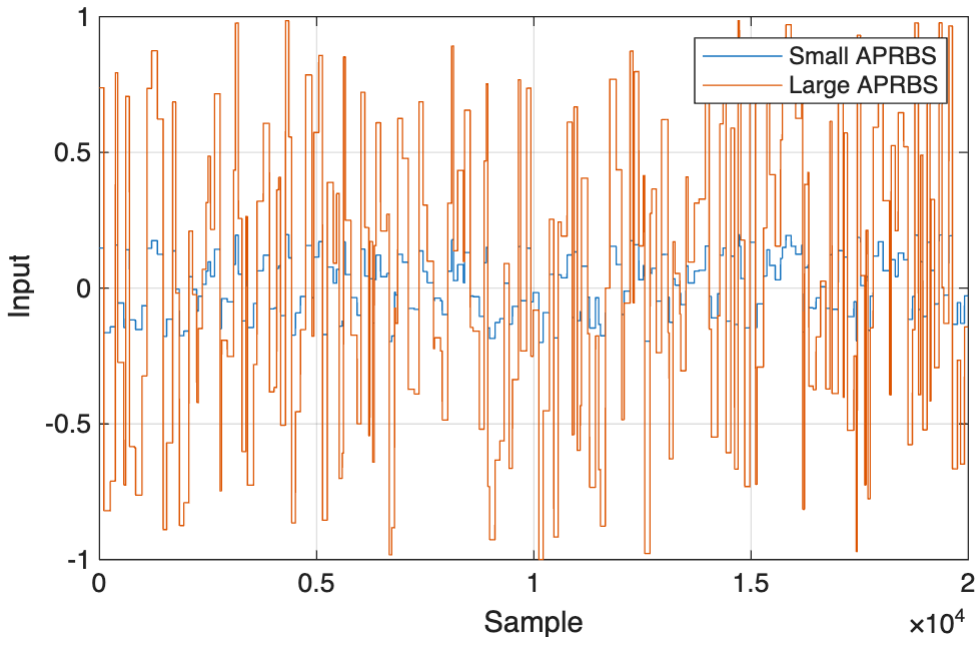
3.2 The dataset is different for each student

Setting up the Training and Validation Dataset

Let's start by generating the APRBS signals, each with either a small or large amplitude and variance. To do this, we use `generate_aprbs` with `holdRange` of 20-150 samples, with amplitudes of ±0.2 and ±1 for the small and large amplitude signal. The chart below shows the inputs generated.

```
N = 20000;
holdRange = [20 150];
uSmall = generate_aprbs(N, holdRange, [-0.2 0.2], params.seed);
uLarge = generate_aprbs(N, holdRange, [-1 1], params.seed);

figure;
plot(1:N, [uSmall uLarge]);
xlabel("Sample"); ylabel("Input"); grid on;
legend("Small APRBS", "Large APRBS");
```



Now let's use the assigned model `sys5.mat` and generate the two outputs using `lsim`. We then get the first half as the training dataset `nTrain` and shown it in the chart below.

```
S = load("sys5.mat", "sys5");
plant = S.sys5;

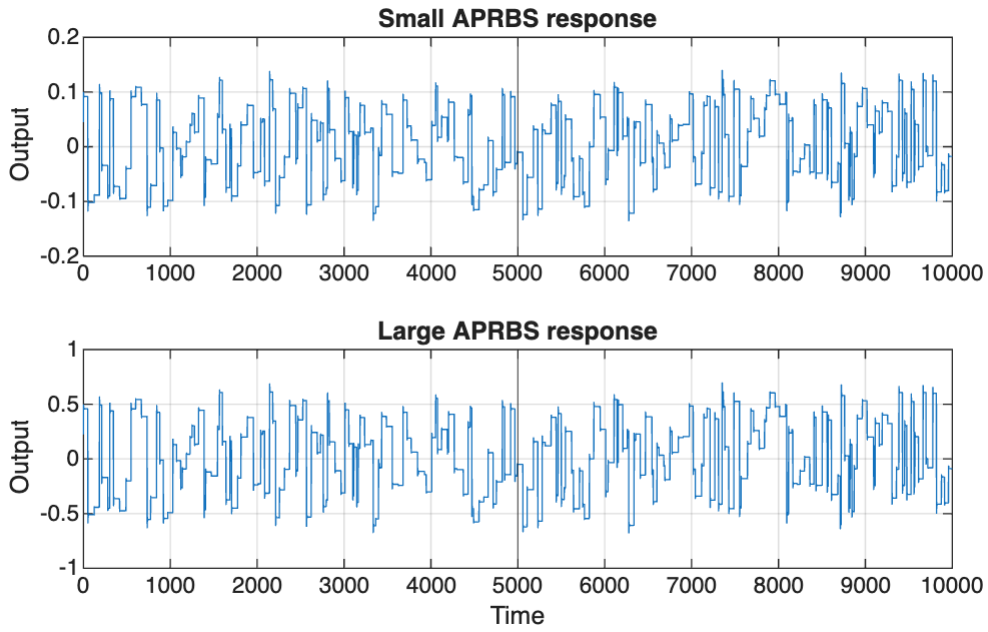
Ts = 0.5
```

```
Ts =
0.5000
```

```
t = (0:N-1)' * Ts;

ySmall = lsim(plant, uSmall, t);
yLarge = lsim(plant, uLarge, t);
nTrain = 10000;

figure;
tiledlayout(2,1);
nexttile; plot(t, ySmall); xline(t(nTrain+1));
title("Small APRBS response"); ylabel("Output"); grid on;
nexttile; plot(t, yLarge); xline(t(nTrain+1));
title("Large APRBS response"); xlabel("Time"); ylabel("Output"); grid on;
```



LSTM

Now let's train an LSTM to predict the output sequence. Helper functions were setup to do this, namely:

- **`normalizeSplitData`** - splits and scales the dataset using training statistics.
- **`fitLSTM`** - uses `trainnet` to train an LSTM model with 32 hidden layers for 25 epochs using Adam and MSE.

First the produced dataset is normalized and split between training and validation. We'll use the training set to train and fit the LSTM. Afterwards, we'll use the validation set to get the predictions and use that to get the needed metrics.

```
function [ZTrainN,ZValidN,mu,sigma] = normalizeSplitData(Z,nTrain)
    ZTrain = Z(1:nTrain,:);
    ZValid = Z(nTrain+1:end,:);
    mu = mean(ZTrain,1);
    sigma = std(ZTrain,0,1);
    ZTrainN = (ZTrain-mu)./sigma;
    ZValidN = (ZValid-mu)./sigma;
end

function net = fitLSTM(UTrain,YTrain,maxEpochs)
    arguments
        UTrain
        YTrain
        maxEpochs = 30
    end
    layers = [
        sequenceInputLayer(size(UTrain,2))
        lstmLayer(32)
        fullyConnectedLayer(size(YTrain,2))
    ];
    opts = trainingOptions("adam", ...
        MaxEpochs=maxEpochs,MiniBatchSize=1,GradientThreshold=1, ...
        InputDataFormats="TCB",TargetDataFormats="TCB",Plots="training-
progress");
    net = trainnet({UTrain},{YTrain},layers,"mse",opts);
end

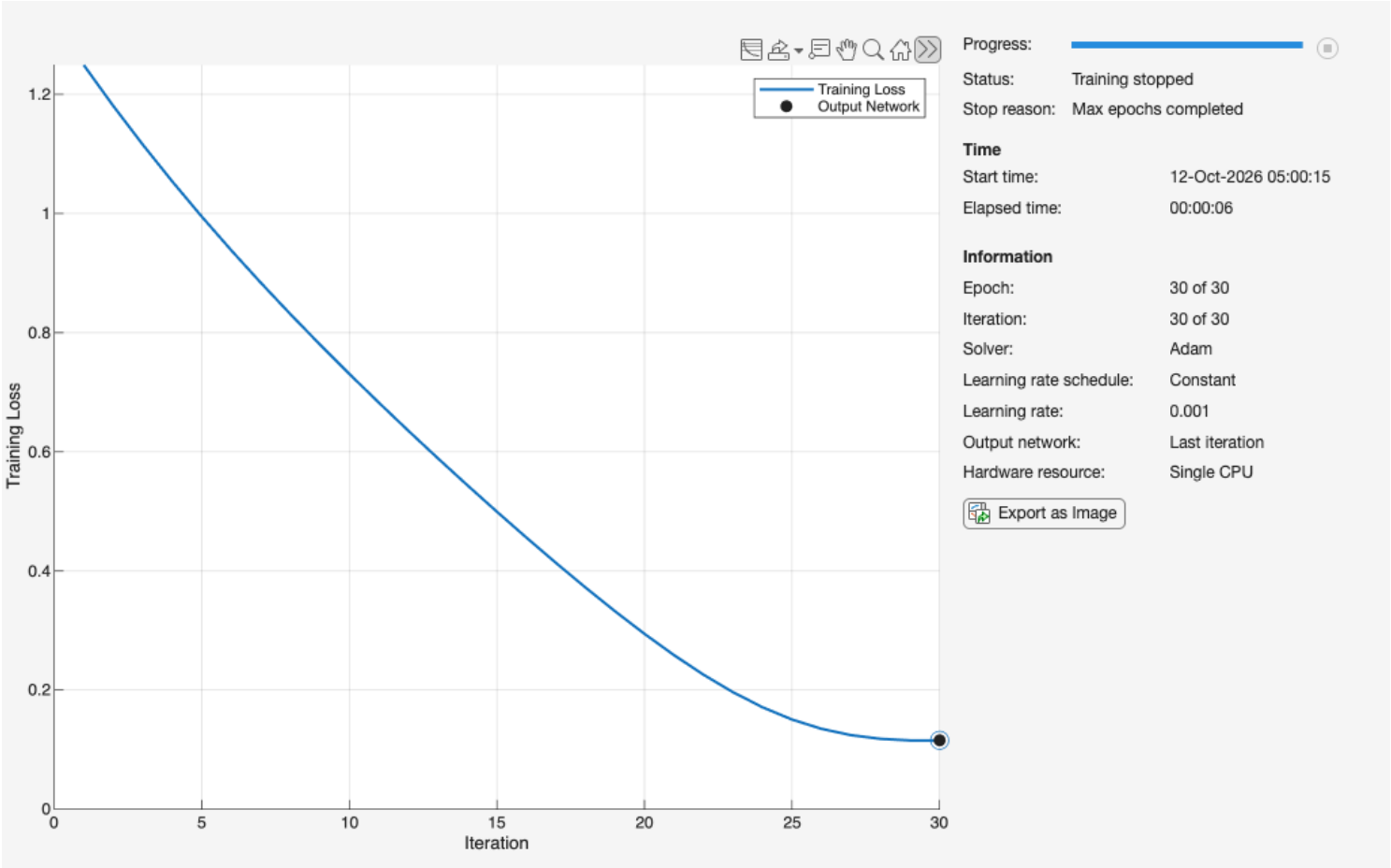
function scores = scoreOutputs(y, yh)
    mse = mean((y-yh).^2,1);
    nmse = goodnessOfFit(yh,y,"NMSE");
    scores = [mse(:), nmse(:), 1-nmse(:)];
end

idx = (nTrain+1:N)';

[uSmallTrainN,uSmallValidN,muUSmall,sdUSmall] =
normalizeSplitData(uSmall,nTrain);
[ySmallTrainN,ySmallValidN,muYSmall,sdYSmall] =
normalizeSplitData(ySmall,nTrain);
[uLargeTrainN,uLargeValidN,muULarge,sdULarge] =
normalizeSplitData(uLarge,nTrain);
[yLargeTrainN,yLargeValidN,muYLarge,sdYLarge] =
normalizeSplitData(yLarge,nTrain);

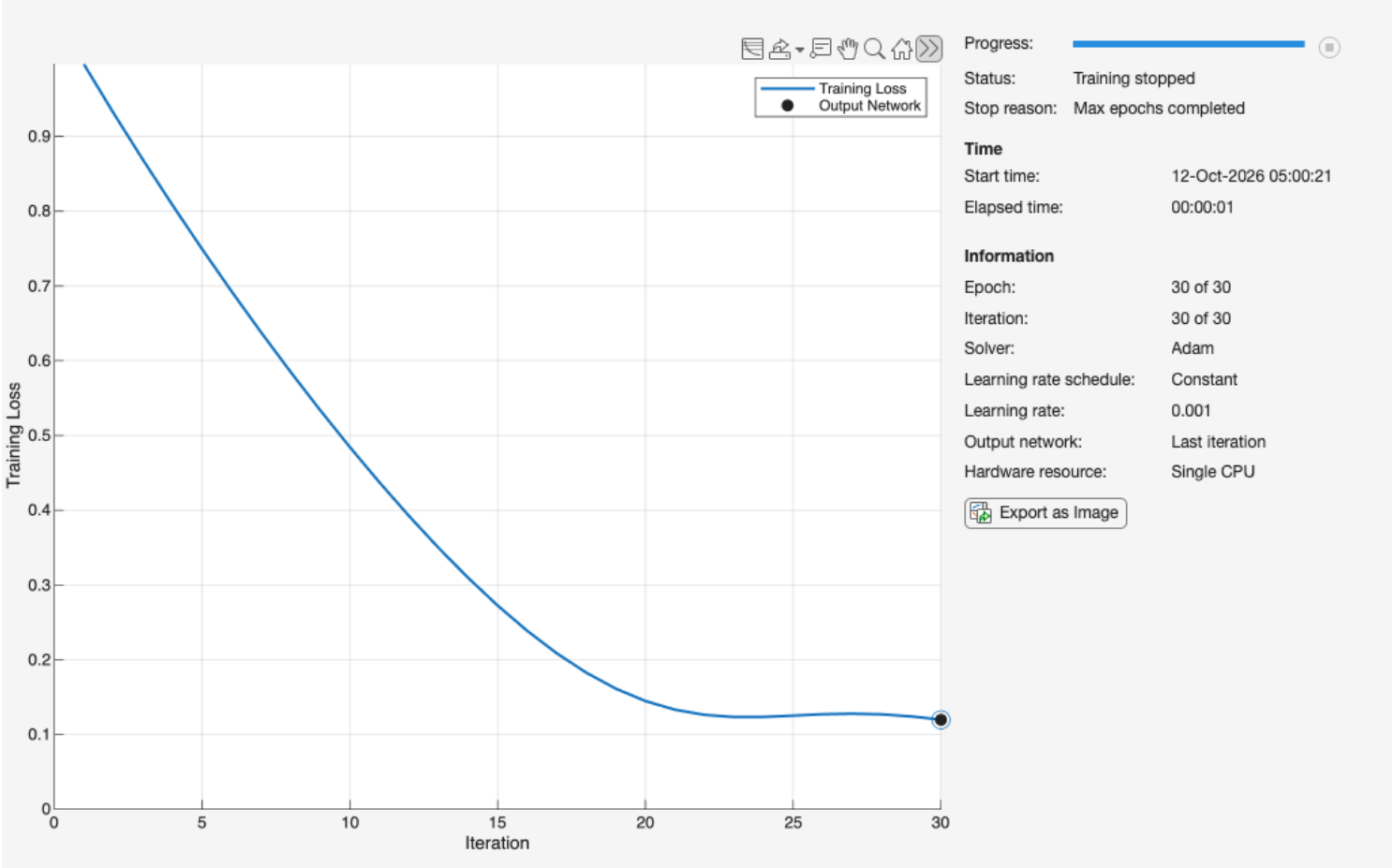
smallLSTM = fitLSTM(uSmallTrainN,ySmallTrainN);
```

```
    Iteration    Epoch    TimeElapsed    LearnRate    TrainingLoss
    _________    _____    ___________    _________    ____________
            1        1       00:00:00        0.001          1.2492
           30       30       00:00:03        0.001         0.11542
Training stopped: Max epochs completed
```




```
largeLSTM = fitLSTM(uLargeTrainN,yLargeTrainN);
```

```
    Iteration    Epoch    TimeElapsed    LearnRate    TrainingLoss
    _________    _____    ___________    _________    ____________
            1        1       00:00:00        0.001         0.99668
           30       30       00:00:01        0.001         0.11983
Training stopped: Max epochs completed
```



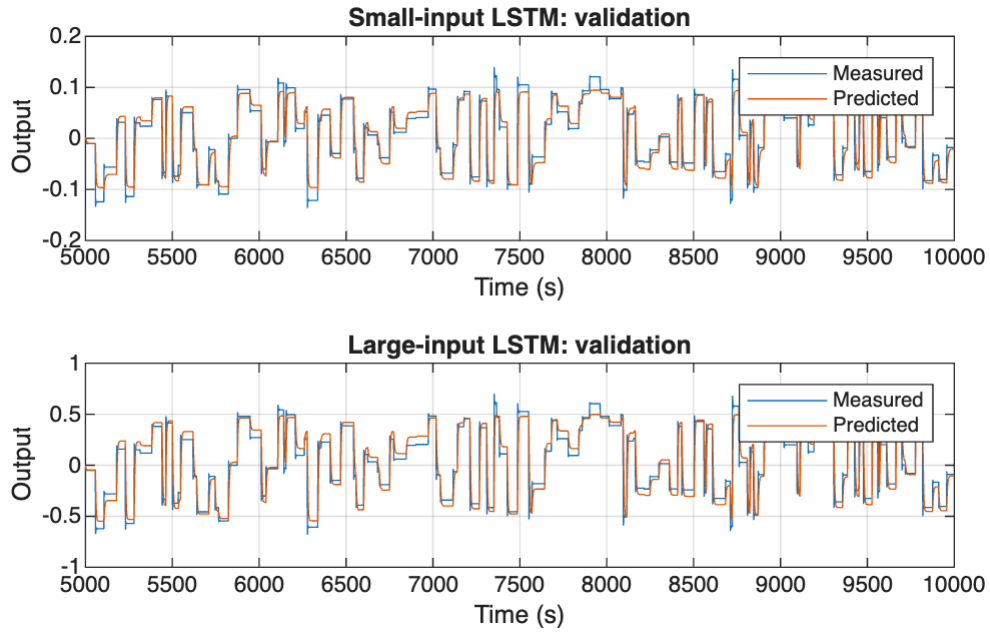

```
% Predict validation inputs only, starting from reset LSTM states.
smallRunN = predict(resetState(smallLSTM),uSmallValidN, ...
    InputDataFormats="TC",OutputDataFormats="TC");
largeRunN = predict(resetState(largeLSTM),uLargeValidN, ...
    InputDataFormats="TC",OutputDataFormats="TC");
smallPrediction = smallRunN.*sdYSmall+muYSmall;
largePrediction = largeRunN.*sdYLarge+muYLarge;
smallScores = scoreOutputs(ySmall(idx,:),smallPrediction);
largeScores = scoreOutputs(yLarge(idx,:),largePrediction);
validationScores = array2table([smallScores;largeScores], ...
    RowNames=["Small","Large"], ...
    VariableNames=["MSE","NMSE","R2"])
```

validationScores = 2×3 table

	MSE	NMSE	R2
1 Small	6.5014e-04	0.1406	0.8594
2 Large	0.0157	0.1360	0.8640

```
figure;
tiledlayout(2,1);
nexttile;
```

```
plot(t(idx),[ySmall(idx) smallPrediction]);
title("Small-input LSTM: validation");
xlabel("Time (s)"); ylabel("Output");
legend("Measured","Predicted"); grid on;
nexttile;
plot(t(idx),[yLarge(idx) largePrediction]);
title("Large-input LSTM: validation");
xlabel("Time (s)"); ylabel("Output");
legend("Measured","Predicted"); grid on;
```



Given that the same hyperparameters are used and the system being linear (meaning a higher-amplitude input should return a higher-amplitude output), it is expected that the performance of both models should pretty much be the same.

NSS Model

Now let's compare with an NSS Model. The following functions are setup to do this:

- **`fitNSS`** - trains an NSS model using MATLAB's default network (two hidden layers of 64 tanh units) with a 2-output history (p) and in 5 epochs .
- **`makeNSSData`** - arranges states and input history.
- **`simulateNSS`** - prepares validation inputs and calls `sim`.

```
function [X,V] = makeNSSData(U,Y,p)
    rows = (p:size(Y,1))';
    X = [];

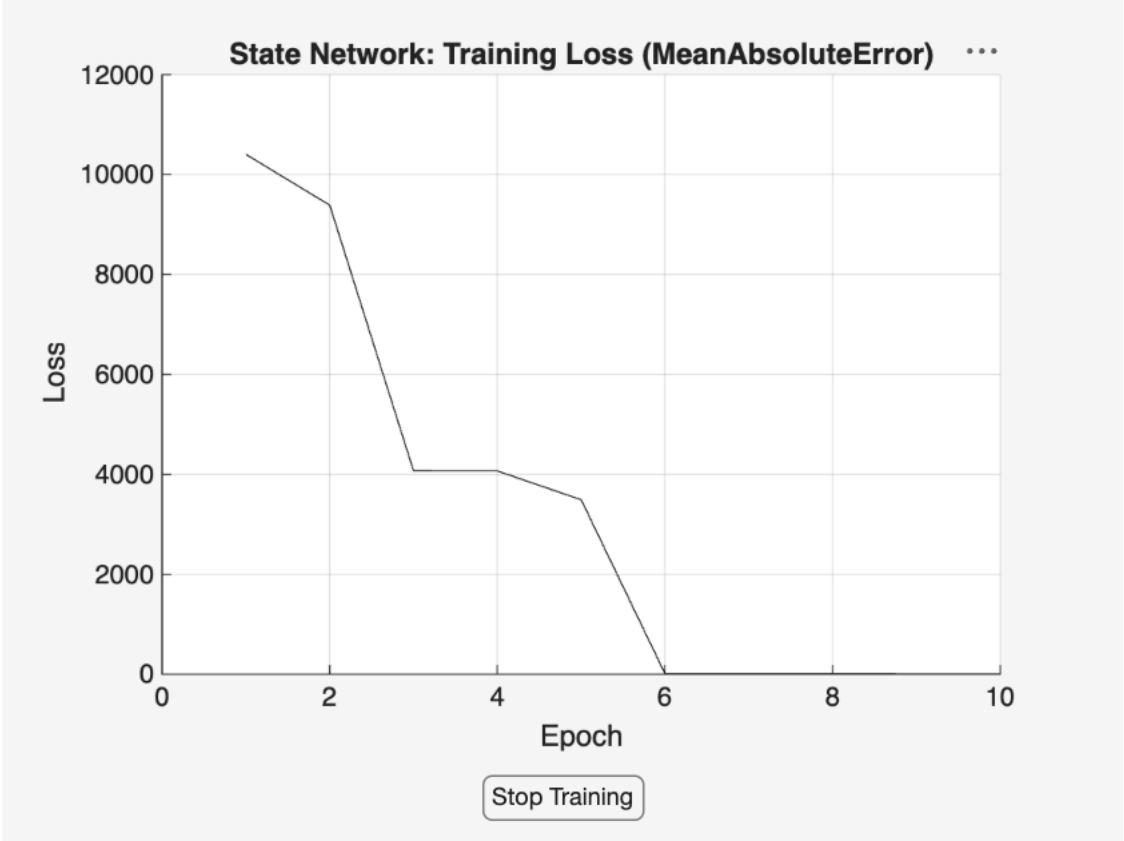
    V = U(min(rows+1,size(U,1)),:);
    for lag = 0:p-1
        X = [X, Y(rows-lag,:)];
```

```
        V = [V, U(rows-lag,:)];
    end
end

function nss = fitNSS(UTrain,YTrain,p,Ts,layerSizes)
    [XTrain,VTrain] = makeNSSData(UTrain,YTrain,p);
    nss0 = idNeuralStateSpace(p*size(YTrain,2), ...
        NumInputs=(p+1)*size(UTrain,2),Ts=Ts);
    if nargin > 4
        nss0.StateNetwork =
createMLPNetwork(nss0,"state",LayerSizes=layerSizes);
    end
    opts = nssTrainingOptions("adam");
    opts.MaxEpochs = 10;
    nss = nlssest(VTrain,XTrain,nss0,opts);
end

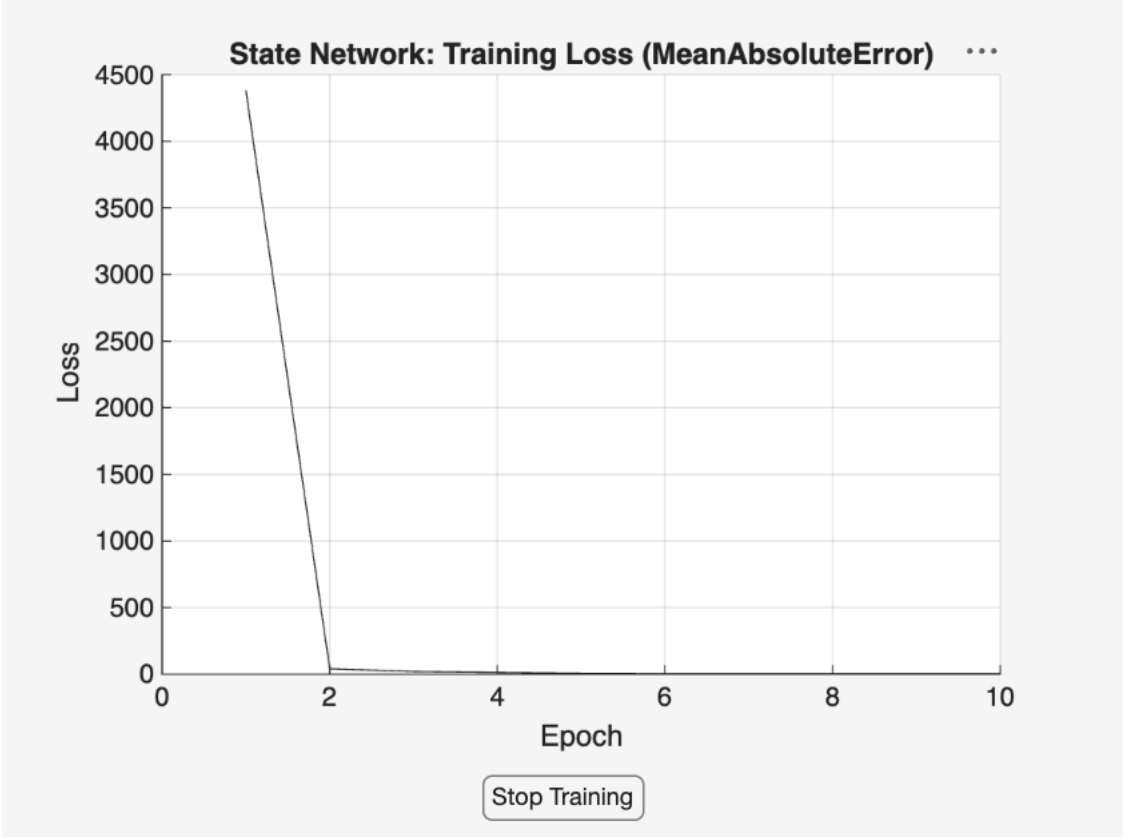
function yhN = simulateNSS(nss,UValid,p,Ts)
    % Reset state and unknown input history to zero (normalized units).
    nx = size(nss,1); % Our NSS outputs are its delay states.
    ny = nx/p;
    U = [zeros(p,size(UValid,2)); UValid];
    rows = (p:size(U,1))';
    V = U(min(rows+1,size(U,1)),:);
    for lag = 0:p-1
        V = [V, U(rows-lag,:)];
    end
    simInput = iddata([],V,Ts);
    simResult = sim(nss,simInput,simOptions(InitialCondition=zeros(nx,1)));
    yhN = simResult.OutputData(2:end,1:ny);
end
```

```
p = 2;
smallNSS = fitNSS(uSmallTrainN,ySmallTrainN,p,Ts);
```




```
Generating estimation report...done.
```

```
largeNSS = fitNSS(uLargeTrainN,yLargeTrainN,p,Ts);
```



```
Generating estimation report...done.
```

```
smallNSSRunN = simulateNSS(smallNSS,uSmallValidN,p,Ts);
largeNSSRunN = simulateNSS(largeNSS,uLargeValidN,p,Ts);
smallNSSPrediction = smallNSSRunN.*sdYSmall+muYSmall;
largeNSSPrediction = largeNSSRunN.*sdYLarge+muYLarge;

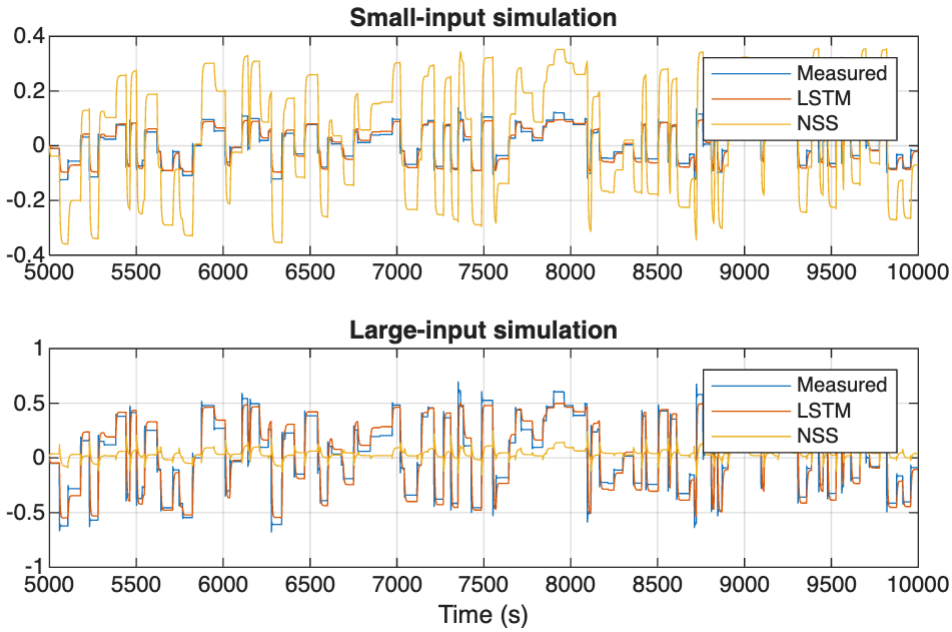
nssPredictions = {smallNSSPrediction; largeNSSPrediction};
nssScores = {scoreOutputs(ySmall(idx,:),smallNSSPrediction); ...
             scoreOutputs(yLarge(idx,:),largeNSSPrediction)};
% Save the trained models and scaling for later questions.
nssModels = {
    struct("nss",smallNSS,"p",p,"muU",muUSmall,"sdU",sdUSmall, ...
        "muY",muYSmall,"sdY",sdYSmall);
    struct("nss",largeNSS,"p",p,"muU",muULarge,"sdU",sdULarge, ...
        "muY",muYLarge,"sdY",sdYLarge)
};

simulationScores =
array2table([smallScores;largeScores;nssScores{1};nssScores{2}], ...
    RowNames=["Small LSTM","Large LSTM","Small NSS","Large NSS"], ...
    VariableNames=["MSE","NMSE","R2"])
```

simulationScores = 4×3 table

	MSE	NMSE	R2
1 Small LSTM	6.5014e-04	0.1406	0.8594
2 Large LSTM	0.0157	0.1360	0.8640
3 Small NSS	0.0238	5.1572	-4.1572
4 Large NSS	0.1069	0.9246	0.0754

```
figure;
tiledlayout(2,1);
nexttile; plot(t(idx),[ySmall(idx,:) smallPrediction nssPredictions{1}]);
title("Small-input simulation"); legend("Measured","LSTM","NSS"); grid on;
nexttile; plot(t(idx),[yLarge(idx,:) largePrediction nssPredictions{2}]);
title("Large-input simulation"); legend("Measured","LSTM","NSS"); grid on;
xlabel("Time (s)");
```



We can see from the table above that LSTM models still fare better than NSS. Then within the 2 NSS models, the large-input NSS model performed better than the small-input NSS model in this run, while the LSTM models showed similar performance across amplitudes.

3.3 Using LSTM to Model an Unknown Linear Model

LSTM

For this section, we introduce noise to our output data as well as doing one-step and five-step predictions to see the models' forecasting abilities (how far can they predict without steering away from the actual output).

For the LSTM models, we re-use `fitLSTM` from the previous section to train the models (small- vs large-input, clean vs noisy output). Afterwards, we'll use `predictLSTM` to do a one-step and five-step prediction using the trained model. Then to compare, we compute the R2 scores of each model.

The function `runLSTMCase` does all these for a given input and output.

```
function plotResiduals(U,Y,idx,yh,t,caseName)
    residual = Y(idx,:)-yh;
    e = residual-mean(residual);
    u = U(idx,:)-mean(U(idx,:));
    maxLag = 25;
    [acf,lags] = xcorr(e,maxLag,"coeff");
    ccf = xcorr(e,u,maxLag,"coeff");

    nexttile;
    plot(t(idx),residual); grid on;
    title(caseName+" - prediction error");
    xlabel("Time (s)"); ylabel("Residual");
    nexttile;
```

```
    stem(lags,acf); grid on;
    title(caseName+" - residual autocorrelation");
    xlabel("Lag (samples)"); ylabel("Correlation");
    nexttile;
    stem(lags,ccf); grid on;
    title(caseName+" - input-residual cross-correlation");
    xlabel("Lag (samples)"); ylabel("Correlation");
end

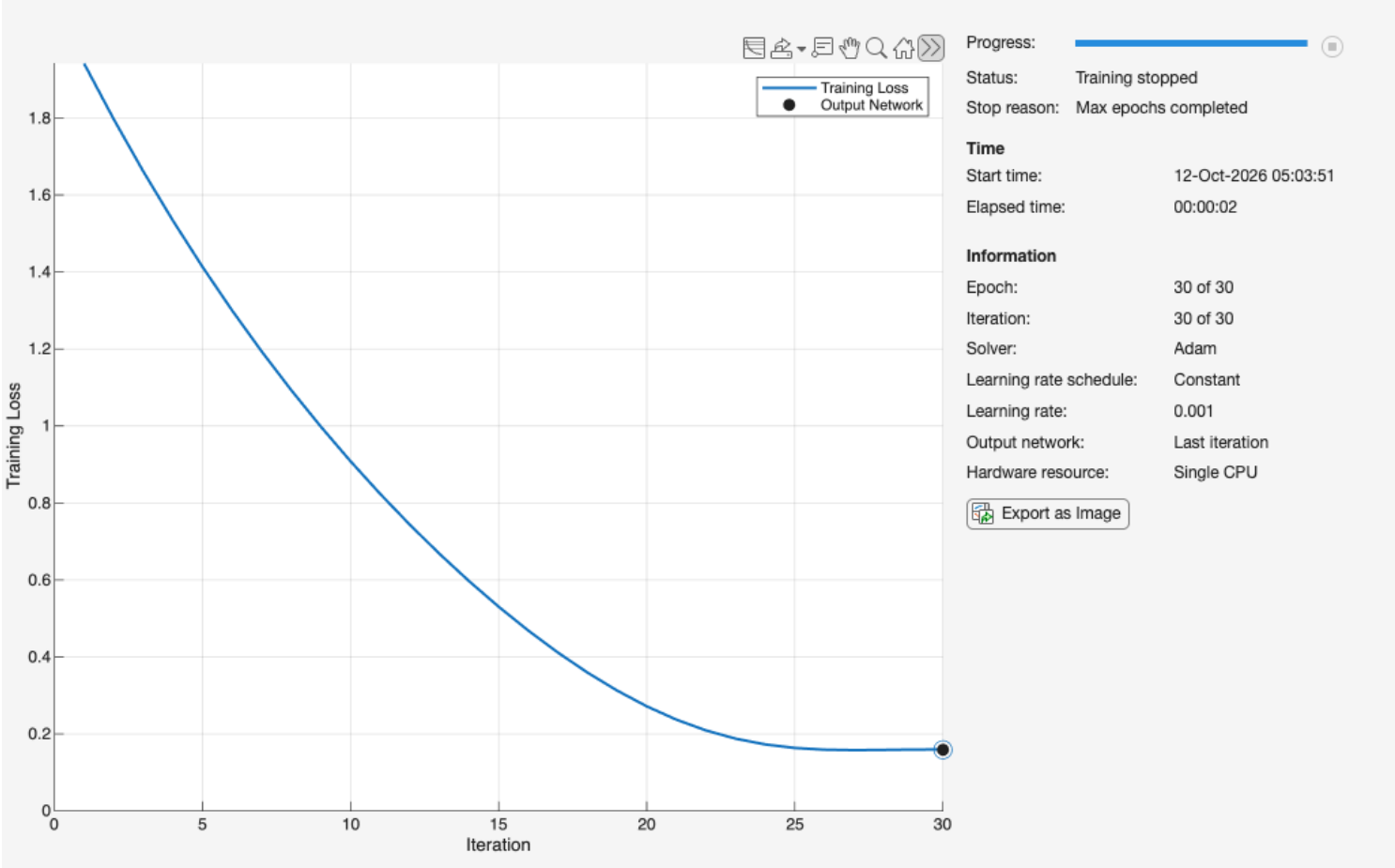
function result = runLSTMCase(U,Y)
    nTrain = 10000;
    [UtrainN,~,muU,sdU] = normalizeSplitData(U,nTrain);
    [YtrainN,~,muY,sdY] = normalizeSplitData(Y,nTrain);
    XTrain = [UtrainN(2:end,:), YtrainN(1:end-1,:)];
    TTrain = YtrainN(2:end,:);
    net = fitLSTM(XTrain,TTrain);
    model = struct("net",net,"muU",muU,"sdU",sdU, ...
        "muY",muY,"sdY",sdY);
    idx = (nTrain+1:size(Y,1))';
    oneStep = predictLSTM(model,U,Y,idx,1);
    fiveStep = predictLSTM(model,U,Y,idx,5);
    scores = [scoreOutputs(Y(idx,:),oneStep); ...
              scoreOutputs(Y(idx,:),fiveStep)];
    result = struct("model",model,"idx",idx,"oneStep",oneStep, ...
        "fiveStep",fiveStep,"scores",scores);
end

function [yh,idx] = predictLSTM(model,U,Y,targetTimes,horizon)
    U = (U-model.muU)./model.sdU;
    Y = (Y-model.muY)./model.sdY;
    idx = targetTimes(:); % Target samples must be in increasing order.
    yh = zeros(numel(idx),size(Y,2));
    measuredNet = resetState(model.net);
    previousOrigin = 0;
    for a = 1:numel(idx)
        origin = idx(a)-horizon;
        % Advance the measured-history state only as far as this origin.
        rows = (previousOrigin+2:origin+1)';
        features = [U(rows,:), Y(rows-1,:)];
        [pred,state] = predict(measuredNet,features, ...
            InputDataFormats="TC",OutputDataFormats="TC");
        measuredNet.State = state;
        pred = pred(end,:); % First forecast: y(origin+1).
        runNet = measuredNet;
        for step = 2:horizon
            feature = [U(origin+step,:), pred];
            [pred,state] = predict(runNet,feature, ...
                InputDataFormats="TC",OutputDataFormats="TC");
            runNet.State = state;
        end
```

```
        yh(a,:) = pred;
        previousOrigin = origin;
    end
    yh = yh.*model.sdY+model.muY;
end

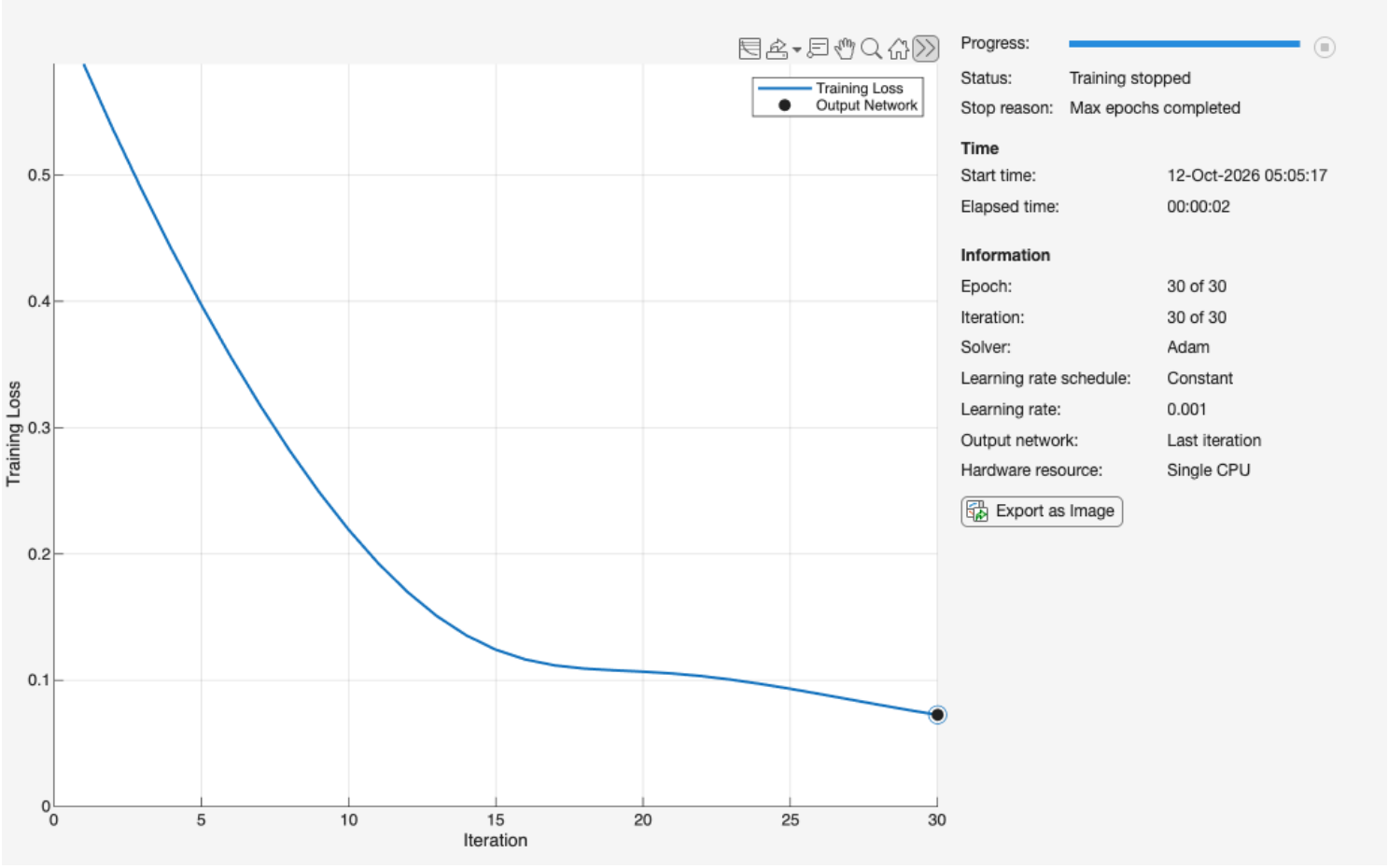
noiseVar = 0.01;
ySmallNoisy = ySmall + sqrt(noiseVar)*randn(size(ySmall));
yLargeNoisy = yLarge + sqrt(noiseVar)*randn(size(yLarge));
smallClean = runLSTMCase(uSmall,ySmall);
```

```
    Iteration    Epoch    TimeElapsed    LearnRate    TrainingLoss
    _________    _____    ___________    _________    ____________
            1        1       00:00:00        0.001          1.9416
           30       30       00:00:02        0.001         0.15949
Training stopped: Max epochs completed
```




```
largeClean = runLSTMCase(uLarge,yLarge);
```

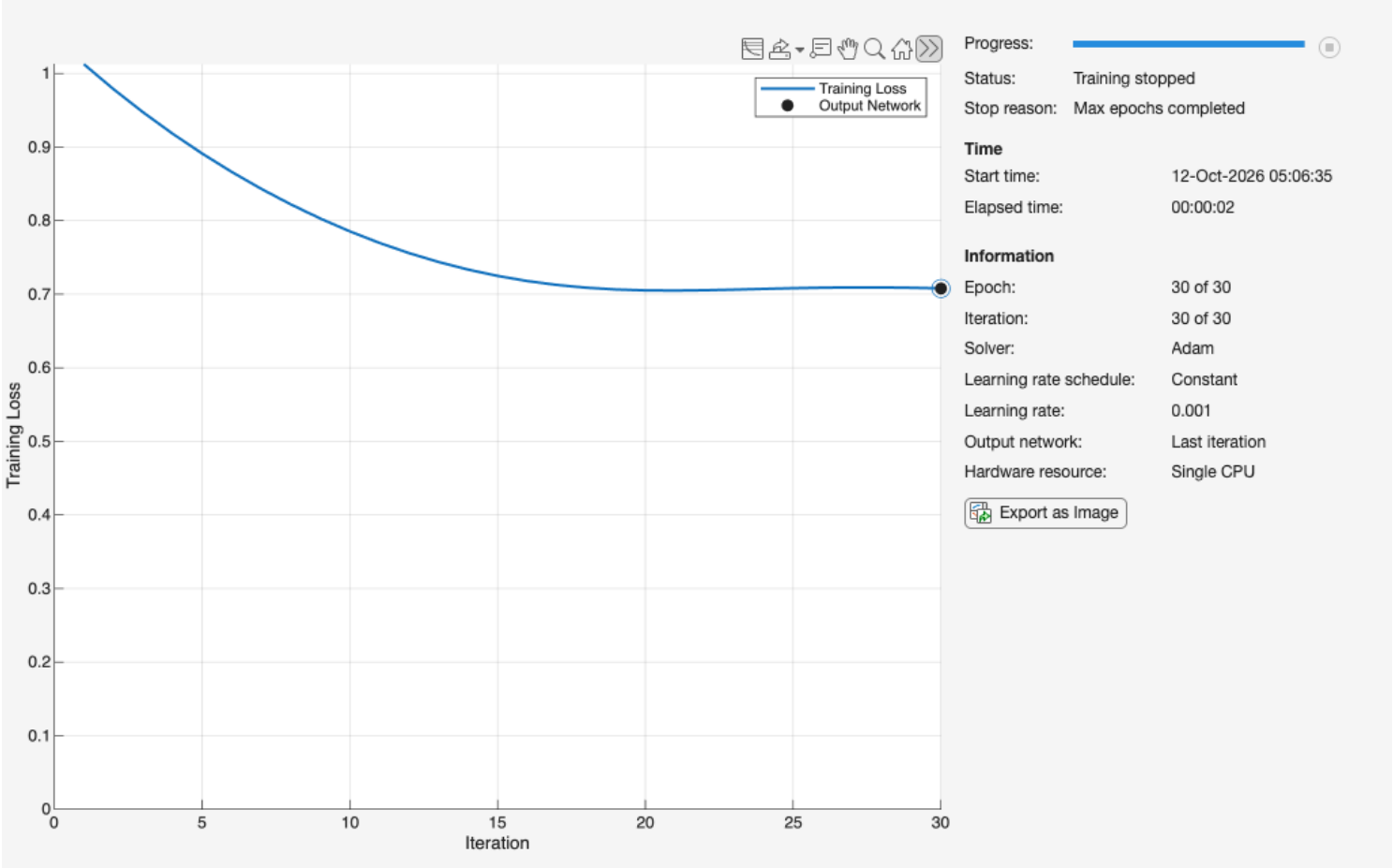
```
    Iteration    Epoch    TimeElapsed    LearnRate    TrainingLoss
    _________    _____    ___________    _________    ____________
            1        1       00:00:00        0.001         0.58784
           30       30       00:00:01        0.001         0.07262
Training stopped: Max epochs completed
```




```
smallNoisy = runLSTMCase(uSmall,ySmallNoisy);
```

```
    Iteration    Epoch    TimeElapsed    LearnRate    TrainingLoss
    _________    _____    ___________    _________    ____________

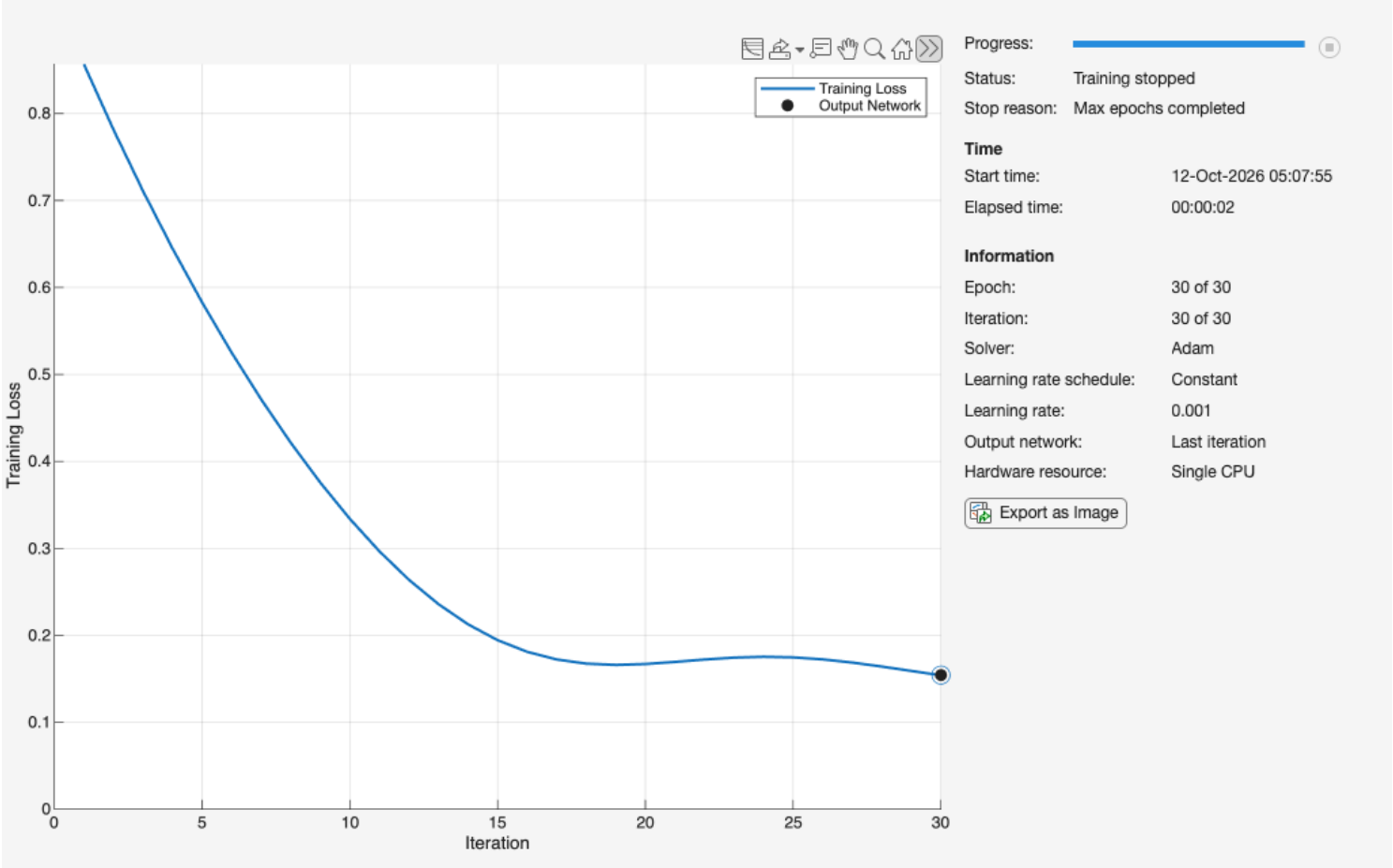
            1        1       00:00:00        0.001          1.0126
           30       30       00:00:01        0.001         0.70788
Training stopped: Max epochs completed
```



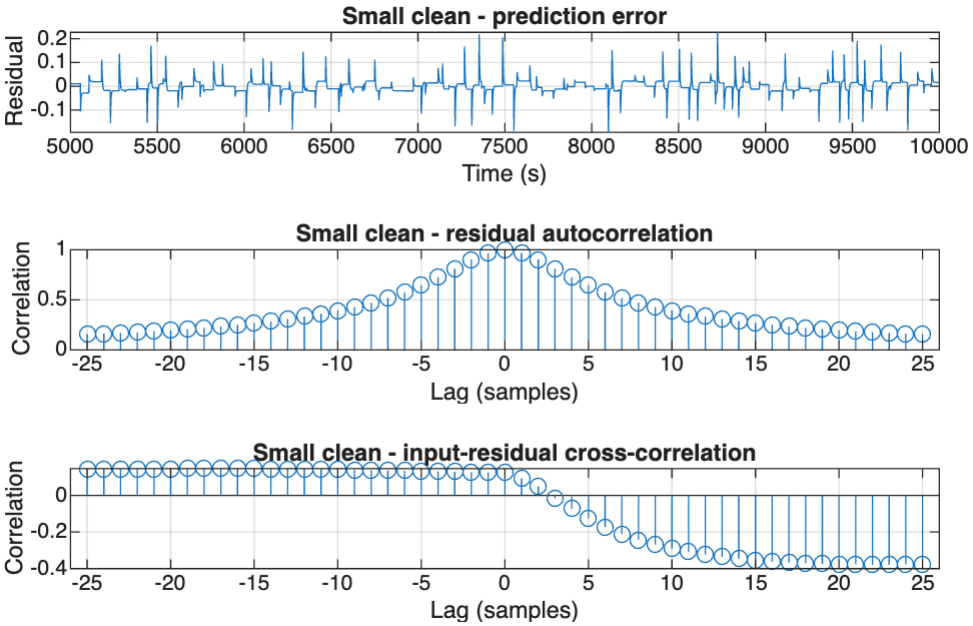

```
largeNoisy = runLSTMCase(uLarge,yLargeNoisy);
```

```
    Iteration    Epoch    TimeElapsed    LearnRate    TrainingLoss
    _________    _____    ___________    _________    ____________

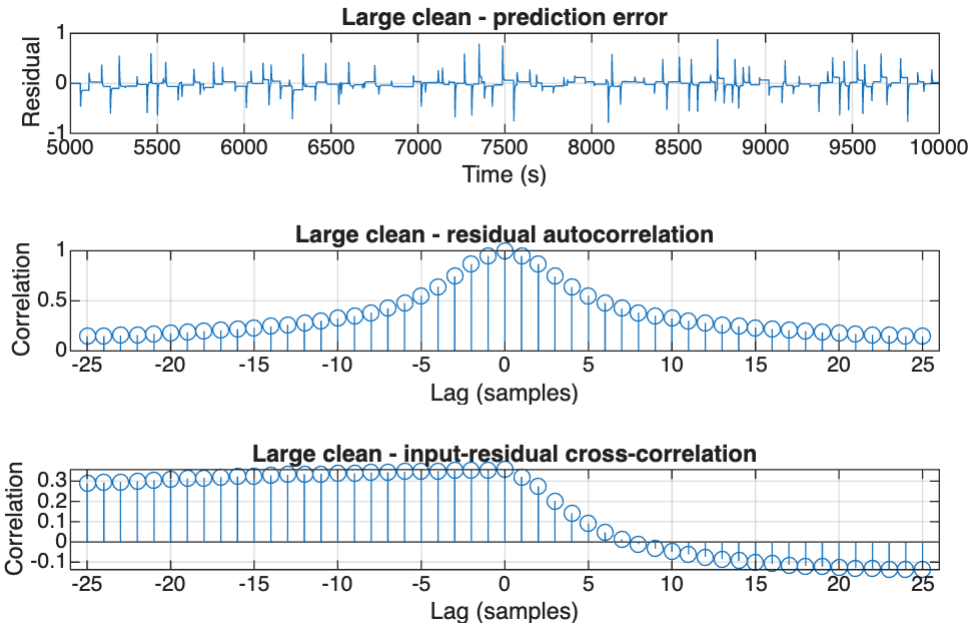
            1        1       00:00:00        0.001         0.85678
           30       30       00:00:01        0.001         0.15396
Training stopped: Max epochs completed
```



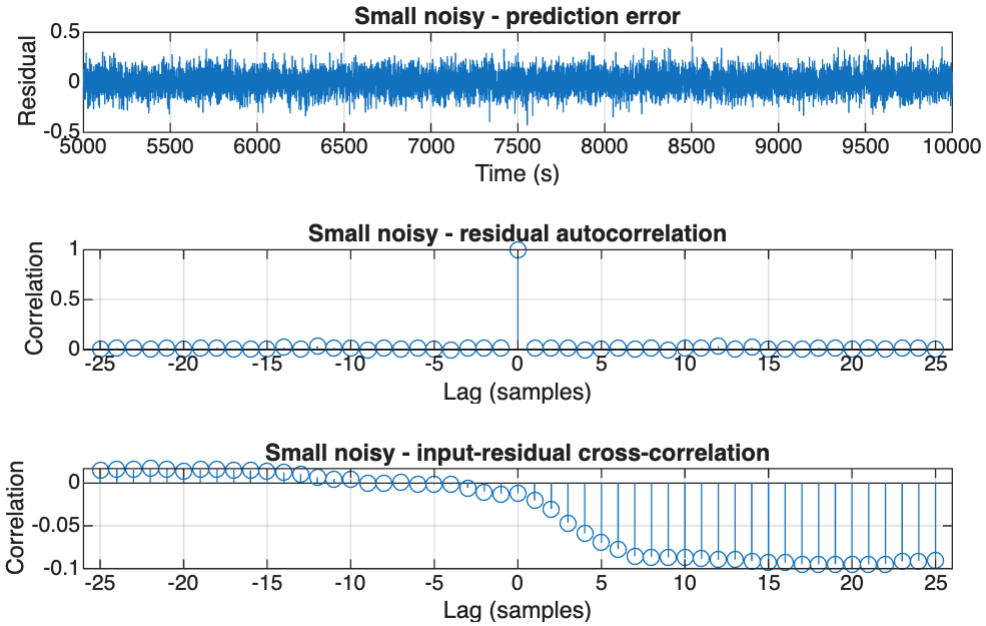

```
cleanLSTMModels = {smallClean.model; largeClean.model};
noisyLSTMModels = {smallNoisy.model; largeNoisy.model};
figure;
tiledlayout(3,1);
plotResiduals(uSmall,ySmall,smallClean.idx,smallClean.oneStep,t,"Small
clean");
```



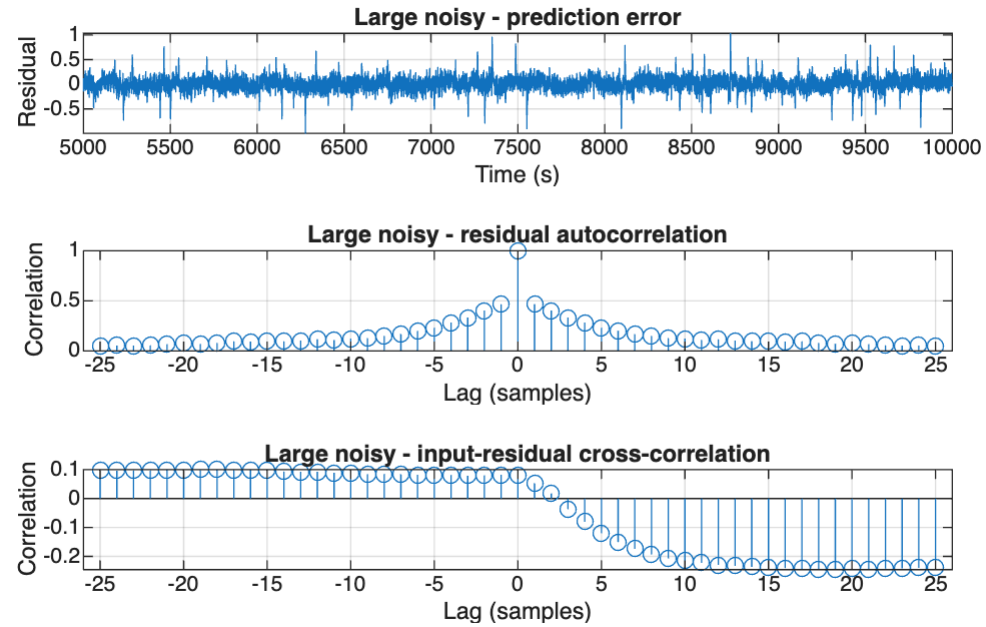

```
figure;
tiledlayout(3,1);
plotResiduals(uLarge,yLarge,largeClean.idx,largeClean.oneStep,t,"Large
clean");
```




```
figure;
tiledlayout(3,1);
plotResiduals(uSmall,ySmallNoisy,smallNoisy.idx,smallNoisy.oneStep,t,"Small
noisy");
```




```
figure;
tiledlayout(3,1);
plotResiduals(uLarge,yLargeNoisy,largeNoisy.idx,largeNoisy.oneStep,t,"Large
noisy");
```



The charts above shows the autocorrelation of the output residual, the prediction error, and the cross-correlation of the input and output residuals for our four models with one-step predictions. Some observations:

- From the prediction error charts, we can see spikes on the clean models, most likely the spots where the signal changes to another value. For the noisy models, we can see the error caused by the noise throughout

the whole sample. We can also observe the same linear change in amplitude of the errors between the small- and large-input models.

- On the residual correlation charts, the clean models has a high correlation at nearby lags and gradually decreases as the gap increases. This is also the same for the large-input noisy model but starting at a smaller correlation just at the nearest lag. For the small-input noisy model correlation is basically at 0 outside lag 0, most likely due to the actual input being drowned out by the noise and hiding the modeling mistakes.
- From the cross-correlation charts, it can be observed that the negative correlations at positive lags indicate that larger past inputs tend to be associated with overprediction and adding noise weakens the input-residual correlations for both amplitude models.

Now comparing the results of the one-step and five-step predictions based on their R2 scores:

```
cleanLSTMScores = array2table([ ...
    smallClean.scores(:,3)'; largeClean.scores(:,3)'], ...
    RowNames=["Small","Large"], ...
    VariableNames=["OneStepR2","FiveStepR2"])
```

`cleanLSTMScores = 2×2 table`

	OneStepR2	FiveStepR2
1 Small	0.8190	0.7582
2 Large	0.9170	0.9028

```
noisyLSTMScores = array2table([ ...
    smallNoisy.scores(:,3)'; largeNoisy.scores(:,3)'], ...
    RowNames=["Small","Large"], ...
    VariableNames=["OneStepR2","FiveStepR2"])
```

`noisyLSTMScores = 2×2 table`

	OneStepR2	FiveStepR2
1 Small	0.3001	0.2926
2 Large	0.8350	0.8145

As expected, one-step predictions performed better than five-step predictions for all models. The large models maintained better accuracy compared to the smaller models on both clean and noisy output. For the clean models, the difference is not large compared to the noisy models given the same linear-only amplitude difference with the small-amplitude noisy model having a far lower R2 score because of the noise variance affecting the outputs more strongly.

NSS Models

We reuse the clean NSS models trained in 3.2 and train two additional models using the same noisy outputs as LSTM. We use `predictNSS` to make one-step and five-step predictions with each trained model, then compute their R2 scores for comparison.

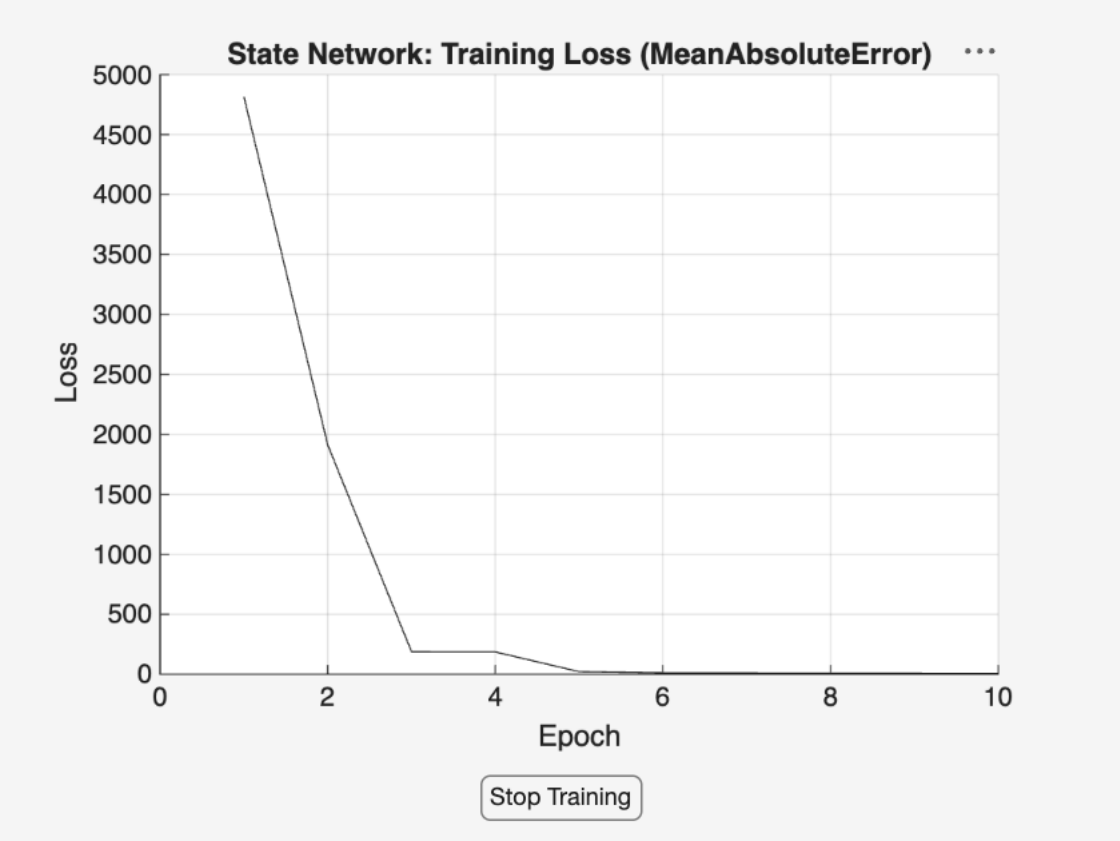
The function `runNSSCase` does these prediction and scoring steps for a given input, output, and trained model, just like `runLSTMCase`.

```
function result = runNSSCase(U,Y,model)
    nTrain = 10000;
    idx = (nTrain+1:size(Y,1))';
    oneStep = predictNSS(model,U,Y,idx,1);
    fiveStep = predictNSS(model,U,Y,idx,5);
    scores = [scoreOutputs(Y(idx,:),oneStep); ...
              scoreOutputs(Y(idx,:),fiveStep)];
    result = struct("model",model,"idx",idx,"oneStep",oneStep, ...
        "fiveStep",fiveStep,"scores",scores);
end

function yh = predictNSS(model,U,Y,idx,horizon)
    U = (U-model.muU)./model.sdU;
    Y = (Y-model.muY)./model.sdY;
    p = model.p;
    ny = size(Y,2);
    yh = zeros(numel(idx),ny);
    for a = 1:numel(idx)
        origin = idx(a)-horizon;
        % Initialize from measured outputs through the forecast origin.
        x = reshape(Y(origin:-1:origin-p+1,:)',[],1);
        for k = origin:idx(a)-1
            v = reshape(U(k+1:-1:k-p+1,:)',[],1);
            [x,~] = evaluate(model.nss,x,v);
        end
        yh(a,:) = x(1:ny)';
    end
    yh = yh.*model.sdY+model.muY;
end

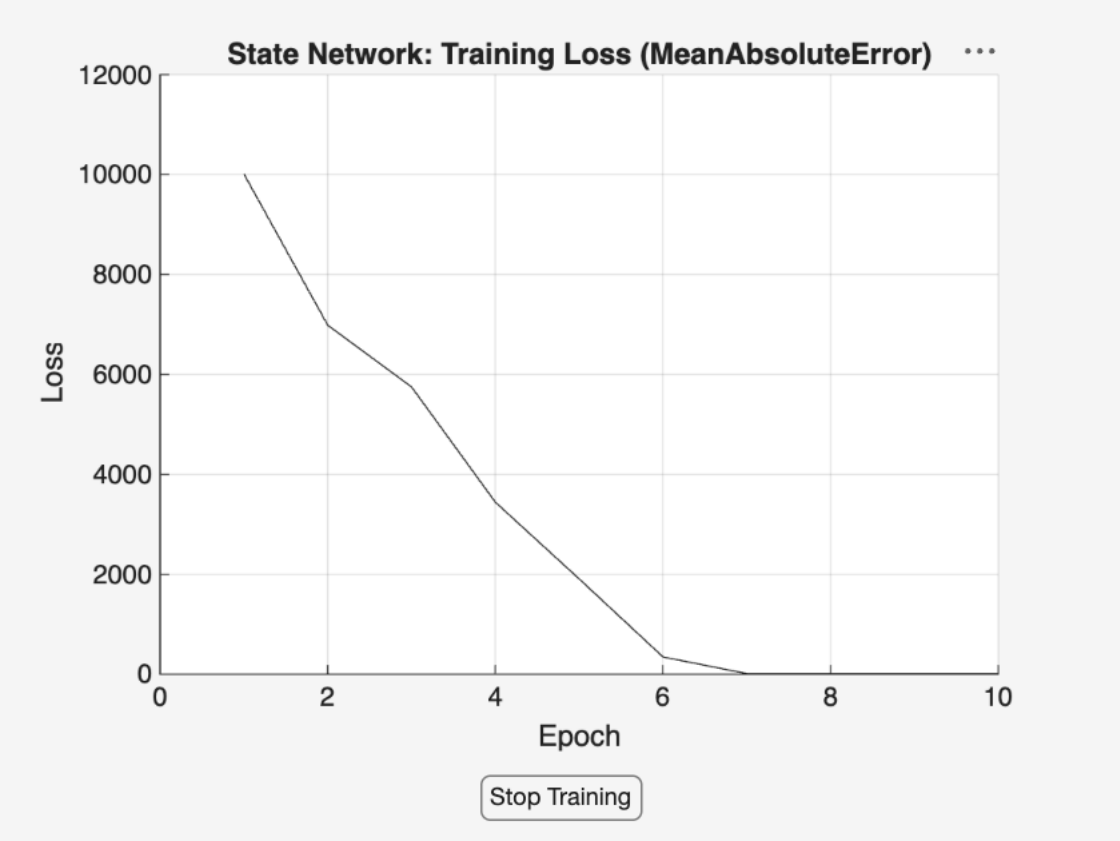
cleanNSSModels = nssModels;

[smallNoisyTrainN,~,muYSmallNoisy,sdYSmallNoisy] = ...
    normalizeSplitData(ySmallNoisy,10000);
[largeNoisyTrainN,~,muYLargeNoisy,sdYLargeNoisy] = ...
    normalizeSplitData(yLargeNoisy,10000);
smallNoisyNSSModel = cleanNSSModels{1};
smallNoisyNSSModel.nss = fitNSS(uSmallTrainN,smallNoisyTrainN,p,Ts);
```




```
Generating estimation report...done.
```

```
smallNoisyNSSModel.muY = muYSmallNoisy;
smallNoisyNSSModel.sdY = sdYSmallNoisy;
largeNoisyNSSModel = cleanNSSModels{2};
largeNoisyNSSModel.nss = fitNSS(uLargeTrainN,largeNoisyTrainN,p,Ts);
```




```
Generating estimation report...done.
```

```
largeNoisyNSSModel.muY = muYLargeNoisy;
largeNoisyNSSModel.sdY = sdYLargeNoisy;
noisyNSSModels = {smallNoisyNSSModel; largeNoisyNSSModel};

smallNSSClean = runNSSCase(uSmall,ySmall,cleanNSSModels{1});
largeNSSClean = runNSSCase(uLarge,yLarge,cleanNSSModels{2});
smallNSSNoisy = runNSSCase(uSmall,ySmallNoisy,noisyNSSModels{1});
largeNSSNoisy = runNSSCase(uLarge,yLargeNoisy,noisyNSSModels{2});

cleanNSSScores = array2table([ ...
    smallNSSClean.scores(:,3)'; largeNSSClean.scores(:,3)'], ...
    RowNames=["Small","Large"], ...
    VariableNames=["OneStepR2","FiveStepR2"])
```

```
cleanNSSScores = 2×2 table
```

	OneStepR2	FiveStepR2
1 Small	0.9663	0.2033
2 Large	0.4158	-0.6828

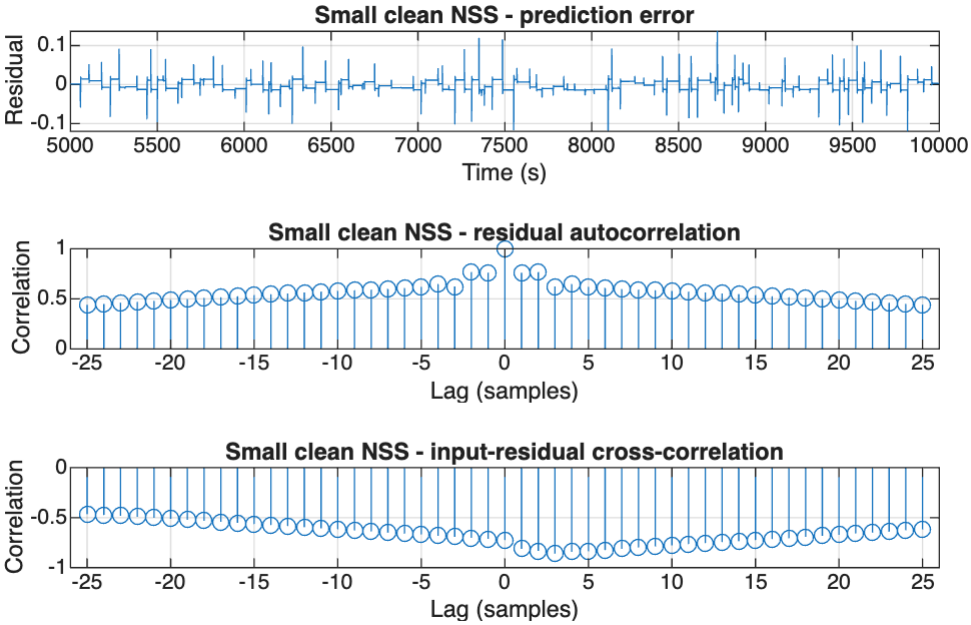
```
noisyNSSScores = array2table([ ...
    smallNSSNoisy.scores(:,3)'; largeNSSNoisy.scores(:,3)'], ...
    RowNames=["Small","Large"], ...
    VariableNames=["OneStepR2","FiveStepR2"])
```

noisyNSSScores = 2×2 table

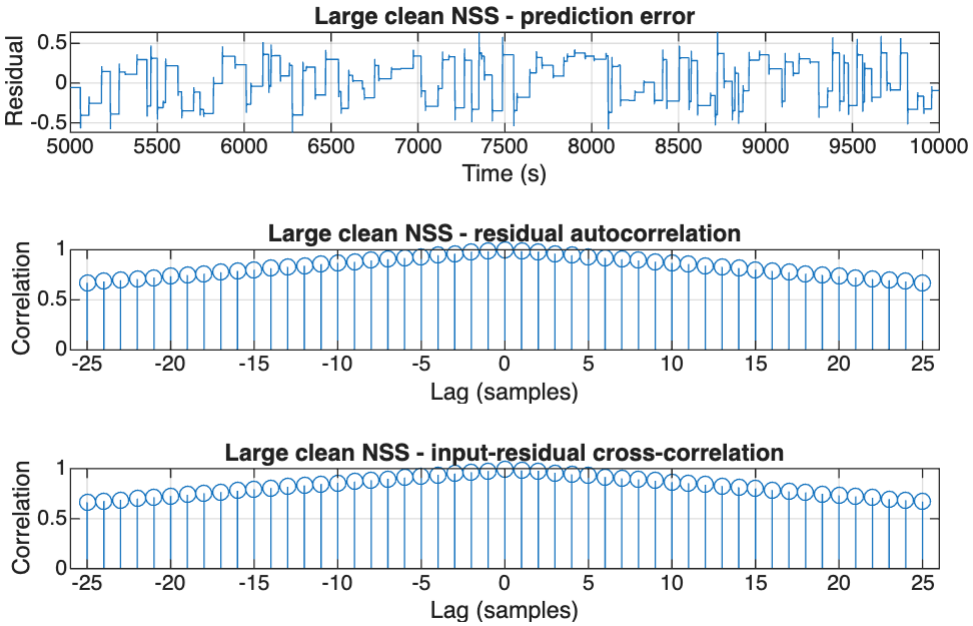
	OneStepR2	FiveStepR2
1 Small	-0.6815	-2.8156
2 Large	-0.0724	-4.3845

One-step predictions performed better than five-step predictions for all NSS models. The clean small-input model achieved the highest accuracy but its performance has a huge drop at five-step which might be caused by the error accumulating more. The noisy models has negative R2 scores showing poor predicting performance and less reliable than the LSTM models.

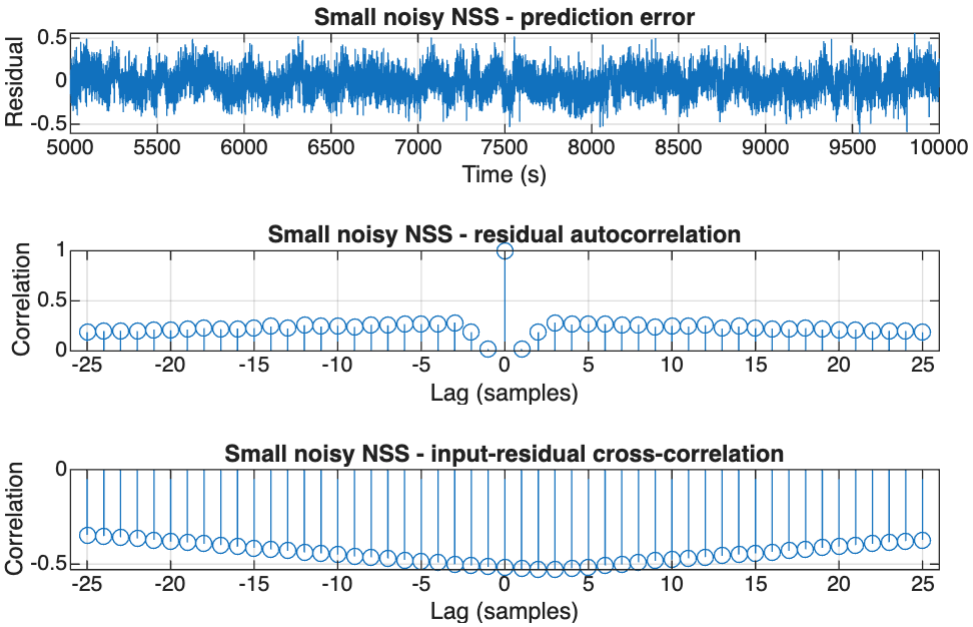
```
figure;
tiledlayout(3,1);
plotResiduals(uSmall,ySmall,smallNSSClean.idx,smallNSSClean.oneStep,t,"Small clean NSS");
```



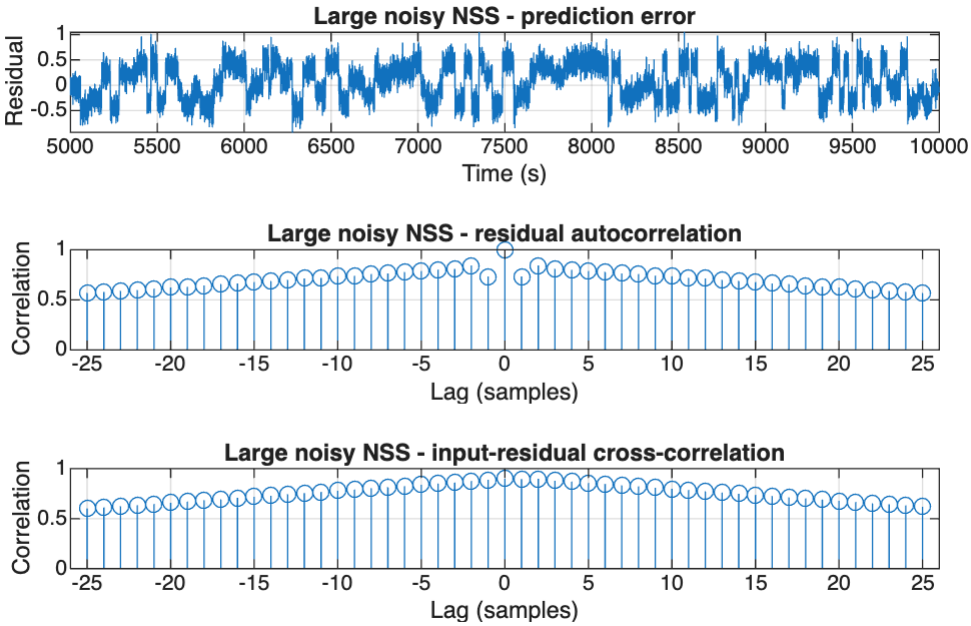

```
figure;
tiledlayout(3,1);
plotResiduals(uLarge,yLarge,largeNSSClean.idx,largeNSSClean.oneStep,t,"Large clean NSS");
```




```
figure;
tiledlayout(3,1);
plotResiduals(uSmall,ySmallNoisy,smallNSSNoisy.idx,smallNSSNoisy.oneStep,t,"S
mall noisy NSS");
```




```
figure;
tiledlayout(3,1);
plotResiduals(uLarge,yLargeNoisy,largeNSSNoisy.idx,largeNSSNoisy.oneStep,t,"L
arge noisy NSS");
```



The clean small-input NSS model achieved better one-step accuracy than LSTM, but its residuals showed stronger patterns and relationships with the input. LSTM performed better in the other three cases and maintained higher accuracy at five steps for every case. Overall, LSTM provided more reliable predictions across noise conditions and prediction horizons in these experiments.

3.4 Newell-Lee Evaporator Model

We start by loading the input and output dataset of the model. Once setup, we can start training different models to see which one can represent the model best.

```
E = load("p2_dat.mat","p2");
small = E.p2{1};
large = E.p2{2};
Ts = 0.5;
nTrain = 5000;
nu = 3;
ny = 3;
inputNames = {'F200';'P100';'F3'};
outputNames = {'L2';'P2';'X2'};
regimeNames = ["Small","Large"];
data = {small,large};
trainData = cell(1,2);
validData = cell(1,2);
for r = 1:2
    U = data{r}(:,1:3);
    Y = data{r}(:,4:6);
    trainData{r} = iddata(Y(1:nTrain,:),U(1:nTrain,:),Ts);
    validData{r} = iddata(Y(nTrain+1:end,:),U(nTrain+1:end,:),Ts);
    trainData{r}.InputName = inputNames;
```

```
    trainData{r}.OutputName = outputNames;
    validData{r}.InputName = inputNames;
    validData{r}.OutputName = outputNames;
end
```

1. NARX

Train NARX models with orders 1 through 10 for both datasets. Use simulation for every order. Choose the highest mean training R2 across L2, P2, and X2, then check its mean validation R2. Plot the measured and predicted outputs for the selected large-amplitude model.

```
qList = 1:10;
resetOptions = simOptions(InitialCondition="z");
resetCompare = compareOptions(InitialCondition="z");
narxModels = cell(1,2);
narxOrders = zeros(1,2);
narxOrderTables = cell(1,2);
for r = 1:2
    zTrain = trainData{r};
    zValid = validData{r};
    candidates = cell(numel(qList),1);
    trainR2 = zeros(numel(qList),ny);
    validR2 = zeros(numel(qList),ny);
    trainInput = iddata([],zTrain.InputData,Ts);
    validInput = iddata([],zValid.InputData,Ts);
    for a = 1:numel(qList)
        q = qList(a);
        orders = [q*ones(ny,ny),q*ones(ny,nu),ones(ny,nu)];
        candidates{a} = nlarx(zTrain,orders);
        yt = sim(candidates{a},trainInput,resetOptions);
        yv = sim(candidates{a},validInput,resetOptions);
        st = scoreOutputs(zTrain.OutputData,yt.OutputData);
        sv = scoreOutputs(zValid.OutputData,yv.OutputData);
        trainR2(a,:) = st(:,3)';
        validR2(a,:) = sv(:,3)';
    end
    narxOrderTables{r} = table(qList',mean(trainR2,2),mean(validR2,2), ...
        VariableNames=["HistoryOrder","MeanTrainR2","MeanValidationR2"]);
    [~,best] = max(mean(trainR2,2));
    narxOrders(r) = qList(best);
    narxModels{r} = candidates{best};
    narxModels{r}.Name = "NARX";
end
```

```
smallOrderTable = narxOrderTables{1}
```

```
smallOrderTable = 10×3 table
```

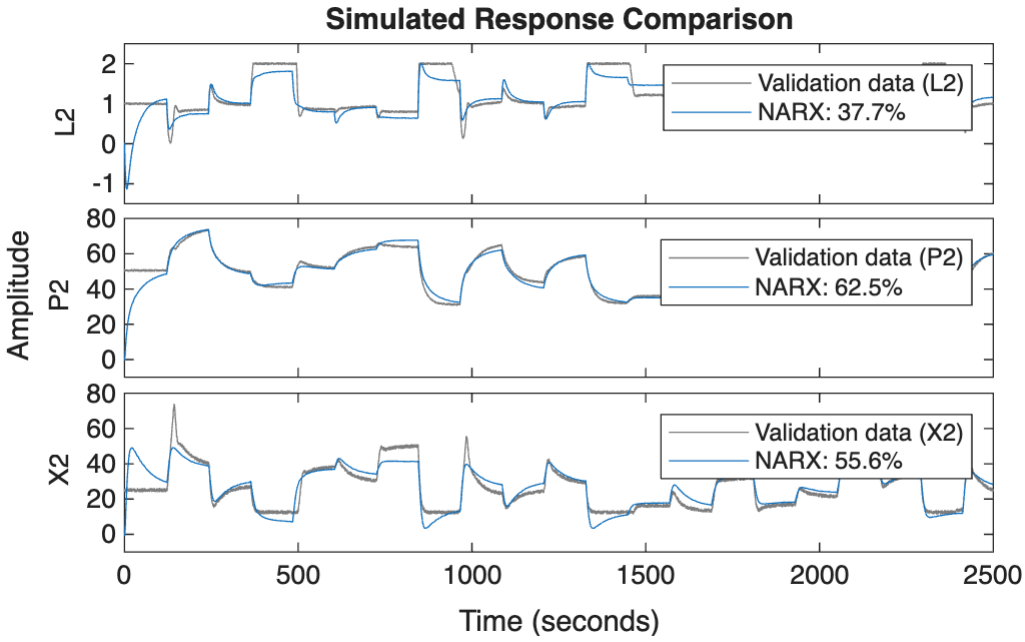
	HistoryOrder	MeanTrainR2	MeanValidationR2
1	1	0.7280	-0.7371
2	2	0.7013	-0.5015
3	3	0.6804	-0.6622
4	4	0.6298	-0.8423
5	5	0.6381	-0.8896
6	6	0.6166	-1.0673
7	7	0.5941	-1.2457
8	8	0.5620	-1.4525
9	9	0.5356	-1.6346
10	10	0.5238	-1.7390

```
largeOrderTable = narxOrderTables{2}
```

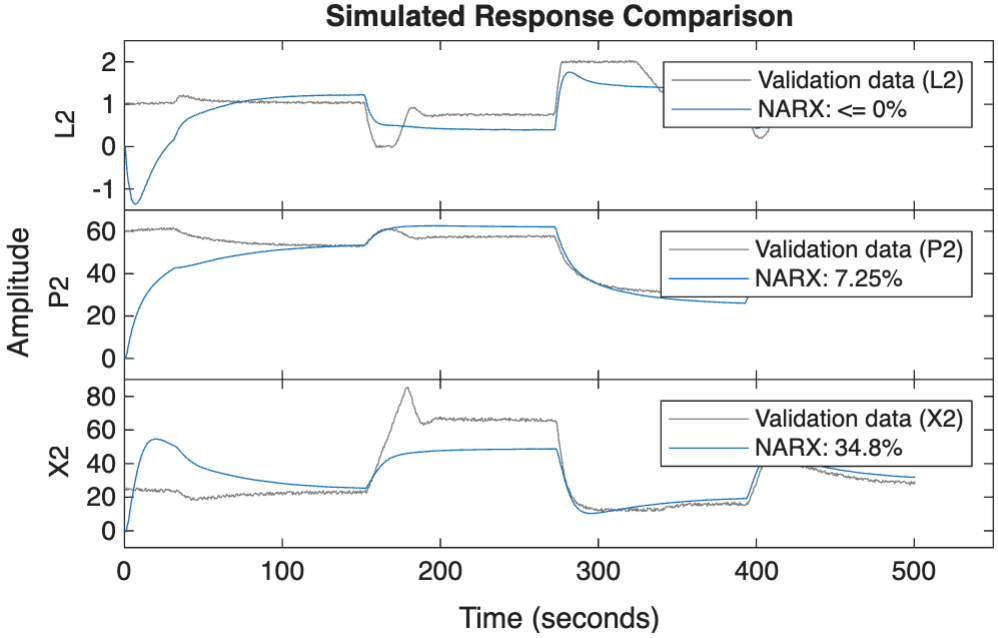
largeOrderTable = 10×3 table

	HistoryOrder	MeanTrainR2	MeanValidationR2
1	1	0.5968	0.0089
2	2	0.6933	-0.0202
3	3	0.6375	-0.0097
4	4	0.6214	-0.0905
5	5	0.6472	-0.0689
6	6	0.6962	-8.7803e-04
7	7	0.7227	0.0552
8	8	0.7435	0.0965
9	9	0.7493	0.0879
10	10	0.7581	0.0879

```
figure;
compare(trainData{2},narxModels{2},resetCompare);
```




```
figure;
compare(validData{2},narxModels{2},resetCompare);
```




```
smallNARXOrder = narxOrders(1)
```

```
smallNARXOrder =
1
```

```
largeNARXOrder = narxOrders(2)
```

```
largeNARXOrder =
10
```

```
smallNARX = narxModels{1};
largeNARX = narxModels{2};
```

For full-simulation, NARX has the highest R2 scores on the 1st order for small-amplitudes and 10th order for large-amplitudes but has low fit percentages with 37-55% for the small-amplitude and 0-35% for the large-amplitude models.

2. Hammerstein-Wiener

Fit a Hammerstein-Wiener model with piecewise-linear input and output maps. Reuse each dataset's selected NARX model and its training/validation R2 scores from 3.4.1.

```
hwPredictionsEvap = cell(1,2);
hwScoresEvap = cell(1,2);
hwMeanTrainR2 = zeros(2,2);
hwMeanValidationR2 = zeros(2,2);
resetOptions = simOptions(InitialCondition="z");
resetCompare = compareOptions(InitialCondition="z");
ordersHW = [2*ones(ny,nu),2*ones(ny,nu),ones(ny,nu)];

for r = 1:2
    zTrain = trainData{r};
    zValid = validData{r};
    hwModels{r} = nlhw(zTrain,ordersHW,idPiecewiseLinear,idPiecewiseLinear);
    trainInput = iddata([],zTrain.InputData,Ts);
    validInput = iddata([],zValid.InputData,Ts);
    yt = sim(hwModels{r},trainInput,resetOptions);
    yv = sim(hwModels{r},validInput,resetOptions);
    st = scoreOutputs(zTrain.OutputData,yt.OutputData);
    sv = scoreOutputs(zValid.OutputData,yv.OutputData);
    hwPredictionsEvap{r} = yv.OutputData;
    hwScoresEvap{r} = sv;
    selectedNARX = narxOrderTables{r}( ...
        narxOrderTables{r}.HistoryOrder == narxOrders(r),:);
    hwMeanTrainR2(r,:) = [selectedNARX.MeanTrainR2,mean(st(:,3))];
    hwMeanValidationR2(r,:) = [selectedNARX.MeanValidationR2,mean(sv(:,3))];
    narxModels{r}.Name = "NARX (order "+narxOrders(r)+")";
    hwModels{r}.Name = "Hammerstein-Wiener";
end
```

```
smallHWTable = table(["NARX (order "+narxOrders(1)+")";"HW"], ...
    hwMeanTrainR2(1,:)',hwMeanValidationR2(1,:)', ...
    VariableNames=["Model","MeanTrainR2","MeanValidationR2"])
```

`smallHWTable = 2×3 table`

	Model	MeanTrainR2	MeanValidationR2
1	"NARX (order 1)"	0.7280	-0.7371

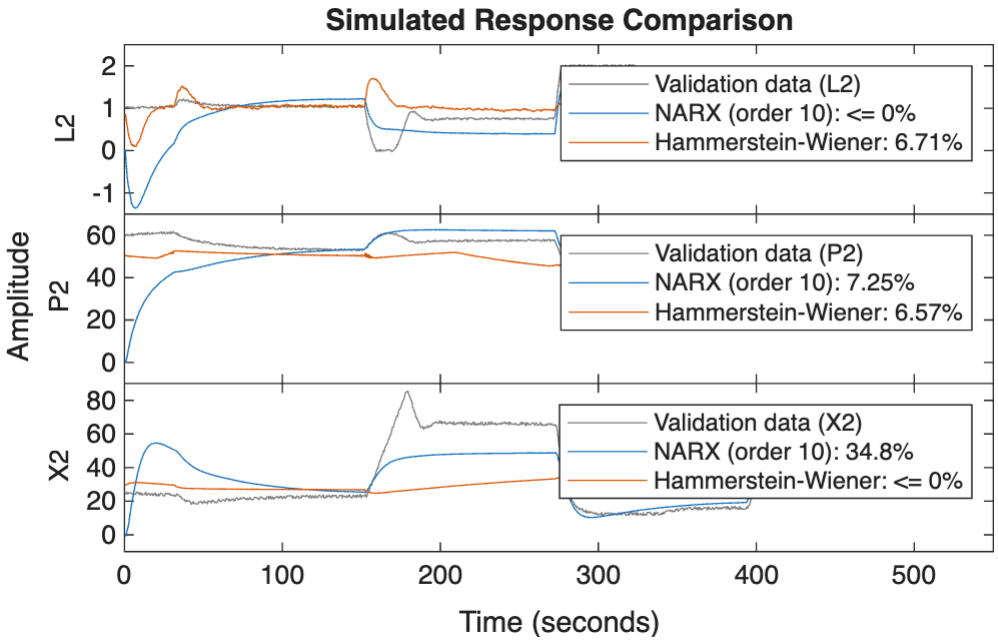
	Model	MeanTrainR2	MeanValidationR2
2	"HW"	0.2942	-0.7280

```
largeHWTable = table(["NARX (order "+narxOrders(2)+")";"HW"], ...
    hwMeanTrainR2(2,:)',hwMeanValidationR2(2,:)', ...
    VariableNames=["Model","MeanTrainR2","MeanValidationR2"])
```

```
largeHWTable = 2×3 table
```

	Model	MeanTrainR2	MeanValidationR2
1	"NARX (order 10)"	0.7581	0.0879
2	"HW"	0.2931	0.0479

```
figure;
compare(validData{2},narxModels{2},hwModels{2},resetCompare);
```



Hammerstein-Wiener is no better than NARX, with validation R2 score in the negatives for small inputs (though HW being higher a bit) but lags on the large-inputs. Overall, NARX performs better on training data and has a modest validation advantage for large inputs. HW offers little overall improvement with the current settings.

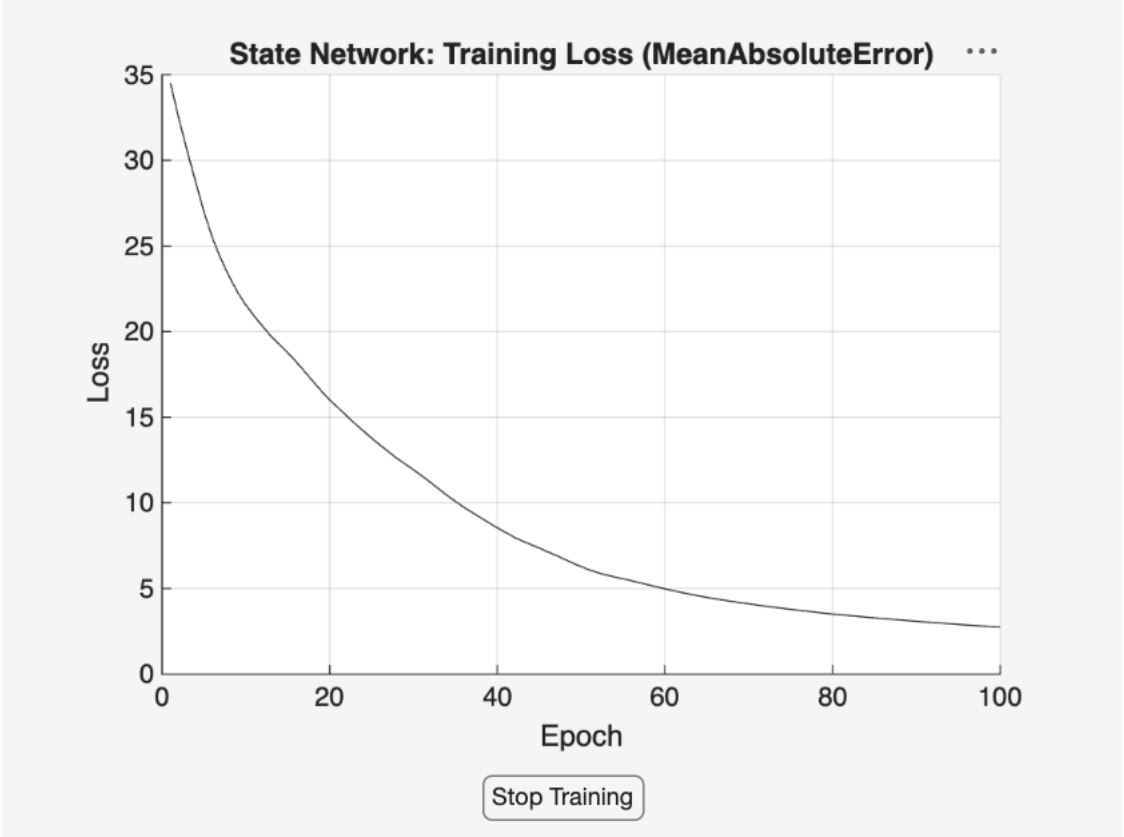
3. Neural state-space

Train a new NSS model for each evaporator dataset and adjust the hyperparameters to achieve high R2 score.

```
nssModelsEvap = cell(1,2);
nssPredictionsEvap = cell(1,2);
nssHWScores = cell(1,2);
nssHWMeanR2 = zeros(2,2);
p = 2;
```

```
evapNSSOptions = nssTrainingOptions("adam");
evapNSSOptions.WindowSize = 32;
evapNSSOptions.Overlap = 16;
evapNSSOptions.MaxEpochs = 100;
for r = 1:2
    U = data{r}(:,1:3);
    Y = data{r}(:,4:6);
    [Ut,Uv,~,~] = normalizeSplitData(U,nTrain);
    [Yt,~,muY,sdY] = normalizeSplitData(Y,nTrain);
    [Xt,Vt] = makeNSSData(Ut,Yt,p);
    nss0 = idNeuralStateSpace(size(Xt,2),NumInputs=size(Vt,2),Ts=Ts);
    nss0.StateNetwork = createMLPNetwork(nss0,"state",LayerSizes=[16 16]);
    nssModelsEvap{r} = nlssest(Vt,Xt,nss0,evapNSSOptions);
    nssPredictionsEvap{r} = simulateNSS(nssModelsEvap{r},Uv,p,Ts).*sdY+muY;
    scores = scoreOutputs(Y(nTrain+1:end,:),nssPredictionsEvap{r});
    nssHWScores{r} = [hwScoresEvap{r};scores];
    nssHWMeanR2(r,:) = [hwMeanValidationR2(r,2),mean(scores(:,3))];
end
```

```
Generating estimation report...done.
Generating estimation report...done.
```




```
smallNSSHWScores = table(["HW";"NSS"],nssHWMeanR2(1,:)', ...
    VariableNames=["Model","MeanValidationR2"])
```

```
smallNSSHWScores = 2×2 table
```

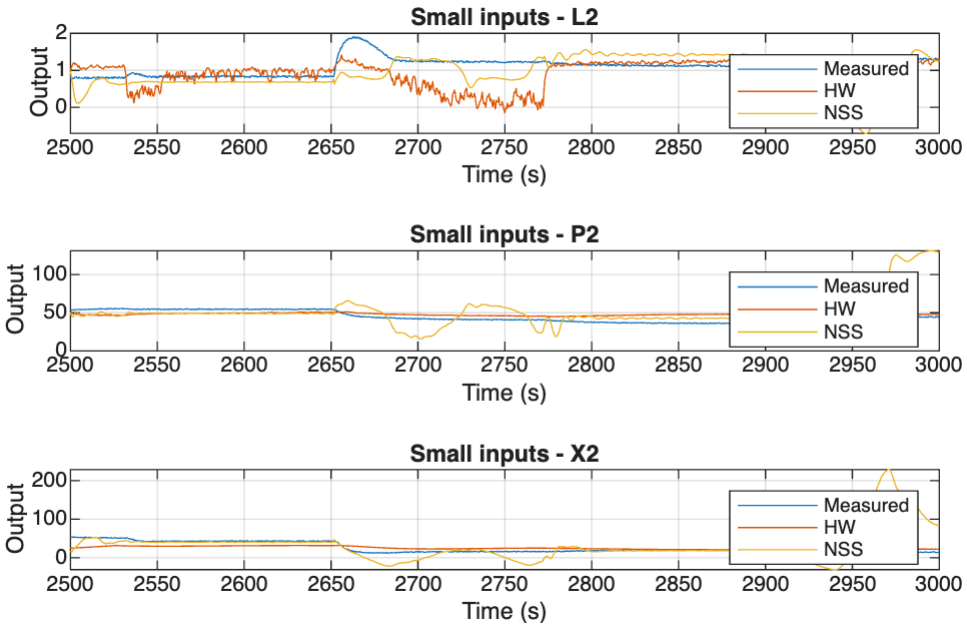
	Model	MeanValidationR2
1	"HW"	-0.7280
2	"NSS"	-7.9908

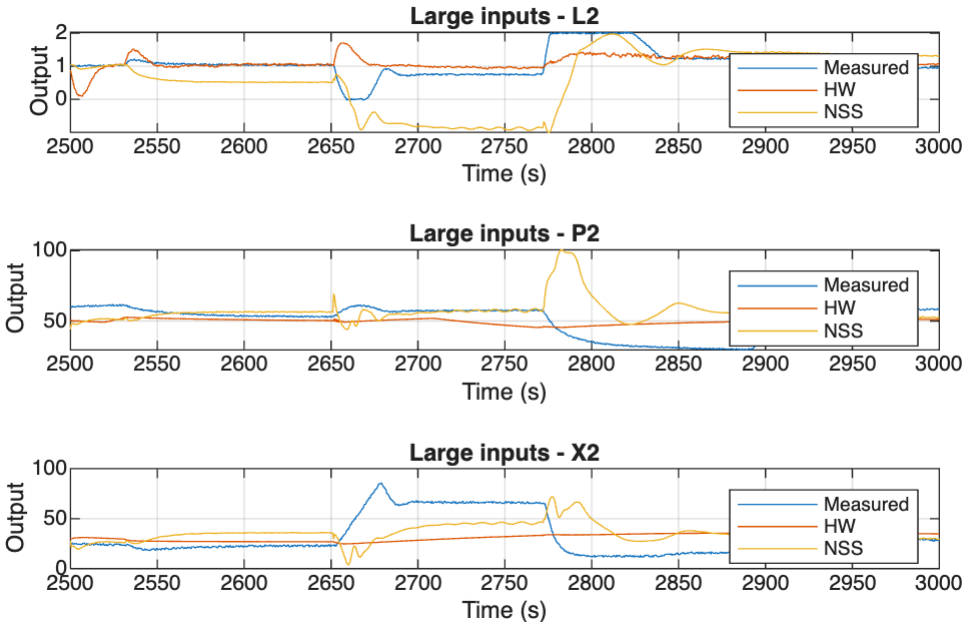
```
largeNSSHWScores = table(["HW";"NSS"],nssHWMeanR2(2,:)', ...
    VariableNames=["Model","MeanValidationR2"])
```

largeNSSHWScores = 2×2 table

	Model	MeanValidationR2
1	"HW"	0.0479
2	"NSS"	-1.6797

```
for r = 1:2
    Yvalid = validData{r}.OutputData;
    tValid = (nTrain:nTrain+size(Yvalid,1)-1)'*Ts;
    figure;
    tiledlayout(ny,1);
    for j = 1:ny
        nexttile;
        plot(tValid,[Yvalid(:,j),hwPredictionsEvap{r}
(:,j),nssPredictionsEvap{r}(:,j)]);
        title(regimeNames(r)+" inputs - "+string(outputNames{j}));
        xlabel("Time (s)"); ylabel("Output"); grid on;
        legend("Measured","HW","NSS");
    end
end
```





After some tweaking, the results are still worse than Hammerstein-Wiener. Some configurations produce a much-higher R2 score for the small-amplitude model but a very low score for large-amplitude models and vice-versa. There might be some configuration that would get a better model for both small- and large-models but I wasn't able to find it.

4. State-space ARX

Fit an SSARX model with six states for each dataset, then compare validation R2, MSE, and simulated outputs with HW and NSS.

```
ssarxModels = cell(1,2);
ssarxPredictions = cell(1,2);
ssarxMeanR2 = zeros(2,3);
ssarxOptions = n4sidOptions(N4Weight="SSARX");
for r = 1:2
    ssarxModels{r} = n4sid(trainData{r},6,ssarxOptions);
    validInput = iddata([],validData{r}.InputData,Ts);
    ssRun = sim(ssarxModels{r},validInput,resetOptions);
    ssarxPredictions{r} = ssRun.OutputData;
end
```

```
ssarxMSETables = cell(1,2);
comparisonModels = ["HW";"NSS";"SSARX"];
for r = 1:2
    Yvalid = validData{r}.OutputData;
    predictions =
{hwPredictionsEvap{r},nssPredictionsEvap{r},ssarxPredictions{r}};
    mse = zeros(3,ny);
    totalVariation = sum((Yvalid-mean(Yvalid,1)).^2,1);
```

```
    for a = 1:3
        errors = Yvalid-predictions{a};
        mse(a,:) = mean(errors.^2,1);
        r2 = 1-sum(errors.^2,1)./totalVariation;
        ssarxMeanR2(r,a) = mean(r2);
    end
    ssarxMSETables{r} = [table(comparisonModels,VariableNames="Model"), ...
        array2table(mse,VariableNames=["MSE_L2","MSE_P2","MSE_X2"])];
end
smallSSARXScores = table(["HW";"NSS";"SSARX"],ssarxMeanR2(1,:)', ...
    VariableNames=["Model","MeanValidationR2"])
```

smallSSARXScores = 3×2 table

	Model	MeanValidationR2
1	"HW"	-0.7280
2	"NSS"	-7.9908
3	"SSARX"	-1.7735

```
largeSSARXScores = table(["HW";"NSS";"SSARX"],ssarxMeanR2(2,:)', ...
    VariableNames=["Model","MeanValidationR2"])
```

largeSSARXScores = 3×2 table

	Model	MeanValidationR2
1	"HW"	0.0479
2	"NSS"	-1.6797
3	"SSARX"	-0.0217

```
smallSSARXMSE = ssarxMSETables{1}
```

smallSSARXMSE = 3×4 table

	Model	MSE_L2	MSE_P2	MSE_X2
1	"HW"	0.1958	47.5020	110.7902
2	"NSS"	0.2936	521.6851	1.9807e+03
3	"SSARX"	0.3646	61.1975	65.5344

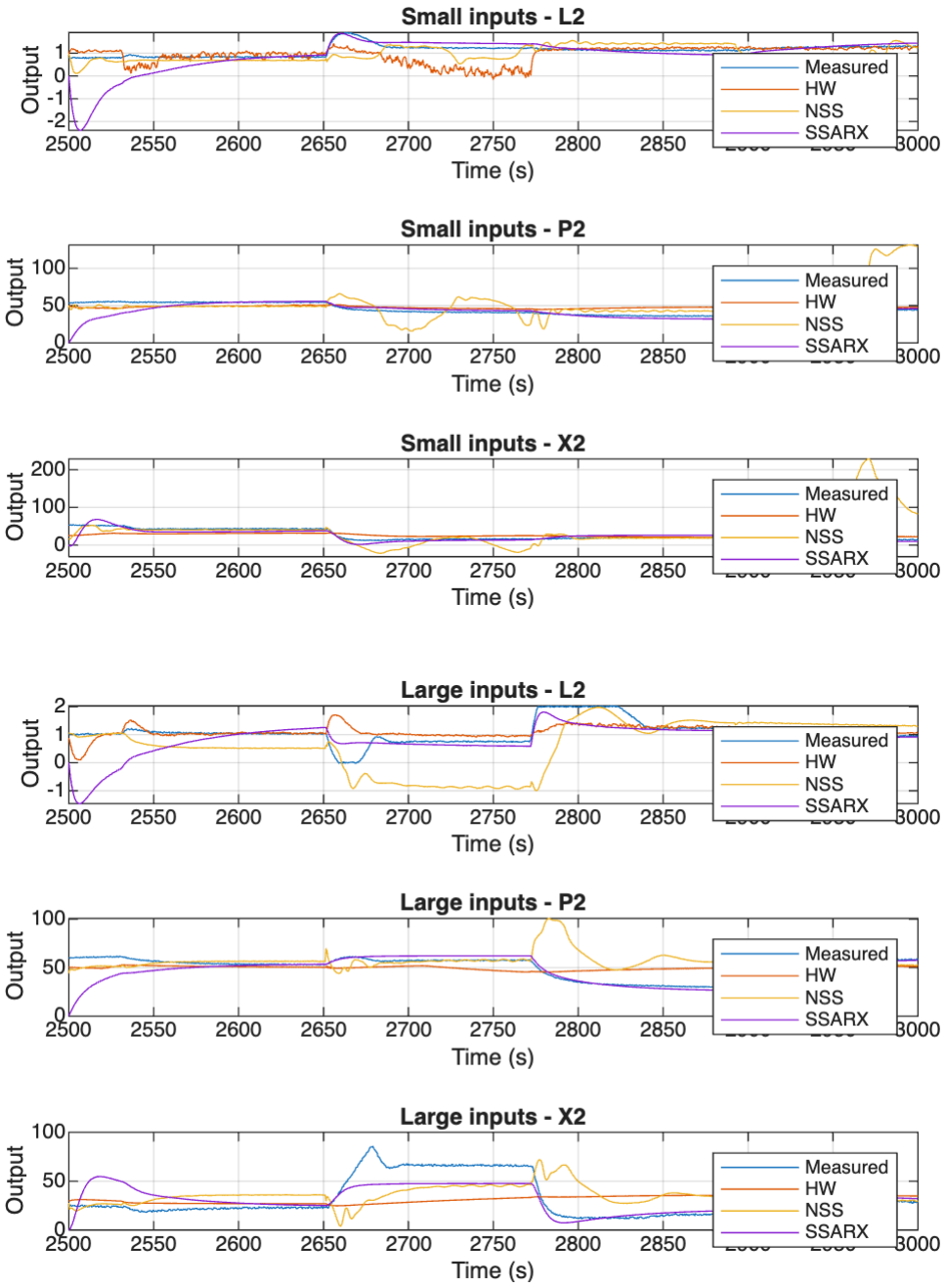
```
largeSSARXMSE = ssarxMSETables{2}
```

largeSSARXMSE = 3×4 table

	Model	MSE_L2	MSE_P2	MSE_X2
1	"HW"	0.1570	94.0279	435.8289
2	"NSS"	0.8146	263.8276	420.8965
3	"SSARX"	0.3351	84.2662	166.7777

```
for r = 1:2
    Yvalid = validData{r}.OutputData;
    tValid = (nTrain:nTrain+size(Yvalid,1)-1)'*Ts;
```

```
    figure;
    tiledlayout(ny,1);
    for j = 1:ny
        nexttile;
        plot(tValid,[Yvalid(:,j),hwPredictionsEvap{r}(:,j), ...
            nssPredictionsEvap{r}(:,j),ssarxPredictions{r}(:,j)]);
        title(regimeNames(r)+" inputs - "+string(outputNames{j}));
        xlabel("Time (s)"); ylabel("Output"); grid on;
        legend("Measured","HW","NSS","SSARX");
    end
end
```



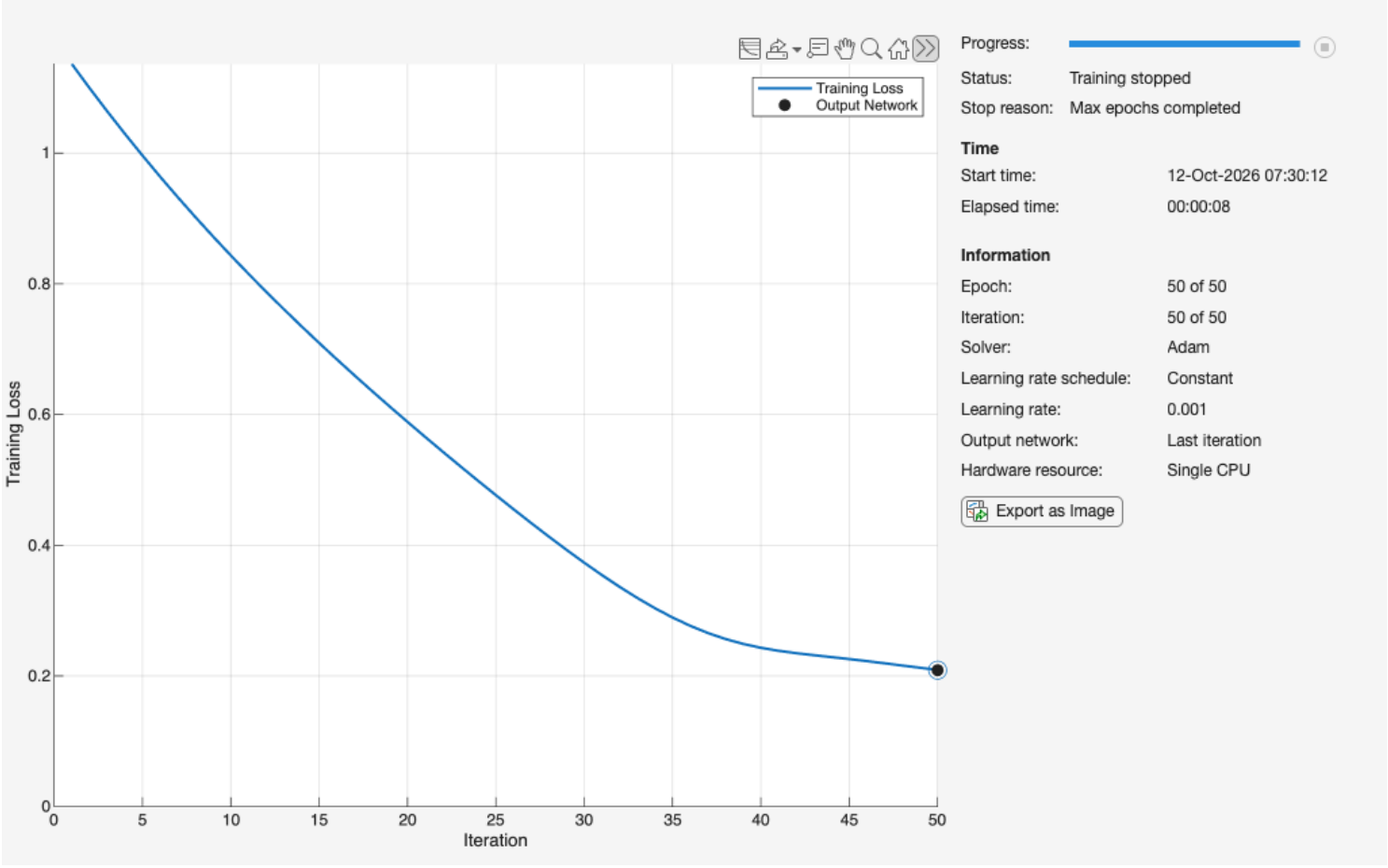
SSARX performed better than NSS for both input amplitudes but not better than HW. But looking at the chart above, we can see SSARX was able to predict the output better than HW but the large startup errors contribute to a far lower R2 score. Checking the MSE score per output (table above), it shows SSARX outperforming HW on X2 and P2 (only on large) so HW might still be the better model to use.

5. LSTM

Train an LSTM for each dataset using fitLSTM with 50 epochs. Compare all five models using mean validation R2 and simulated outputs.

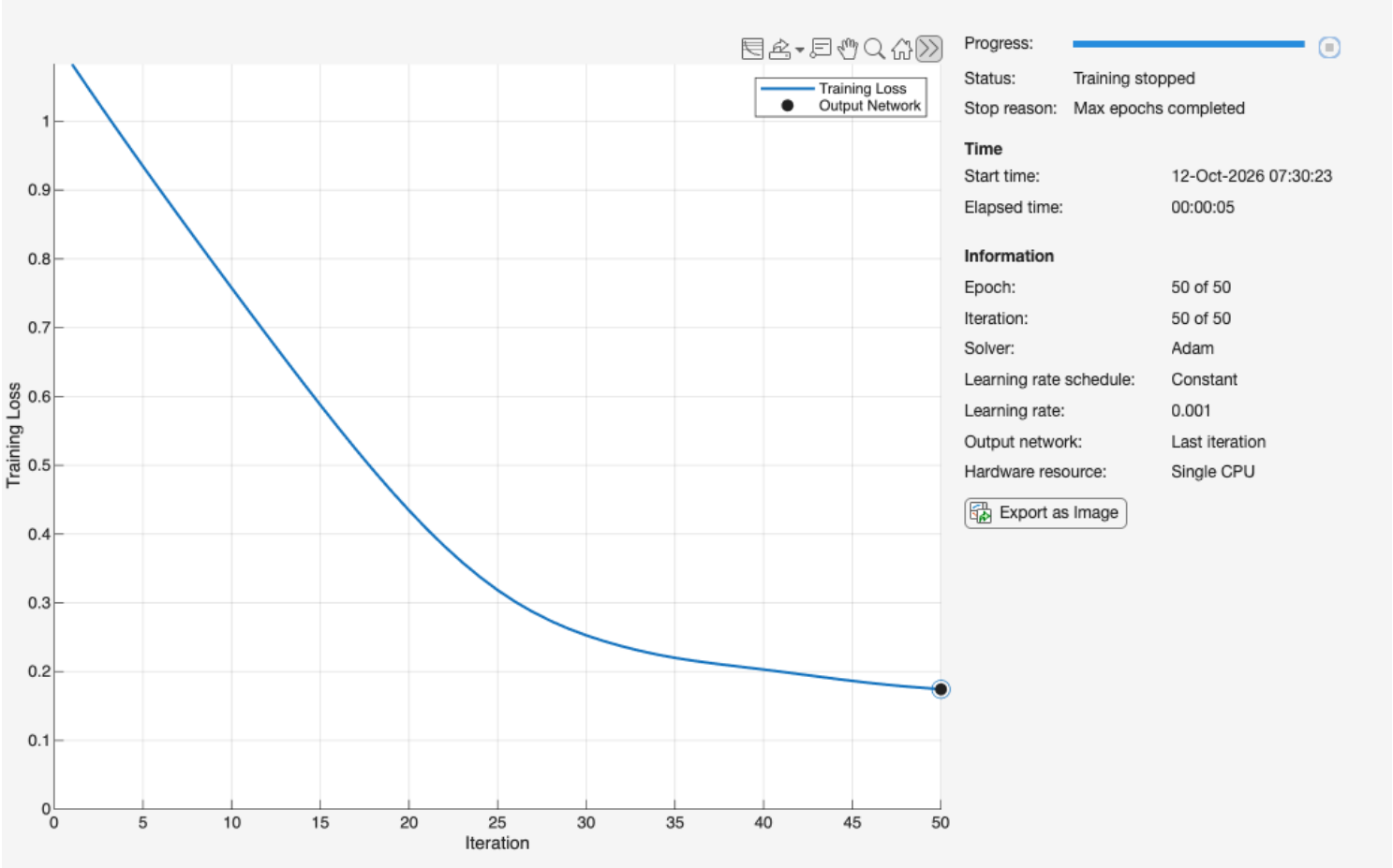
```
lstmModelsEvap = cell(1,2);
lstmPredictionsEvap = cell(1,2);
for r = 1:2
    U = data{r}(:,1:3);
    Y = data{r}(:,4:6);
    [Ut,Uv,~,~] = normalizeSplitData(U,nTrain);
    [Yt,~,muY,sdY] = normalizeSplitData(Y,nTrain);
    lstmModelsEvap{r} = fitLSTM(Ut,Yt,50);
    yhN = predict(resetState(lstmModelsEvap{r}),Uv, ...
        InputDataFormats="TC",OutputDataFormats="TC");
    lstmPredictionsEvap{r} = yhN.*sdY+muY;
end
```

```
    Iteration    Epoch    TimeElapsed    LearnRate    TrainingLoss
    _________    _____    ___________    _________    ____________
            1        1       00:00:01        0.001          1.1363
           50       50       00:00:07        0.001         0.20927
Training stopped: Max epochs completed
```




```
    Iteration    Epoch    TimeElapsed    LearnRate    TrainingLoss
    _________    _____    ___________    _________    ____________

            1        1       00:00:00        0.001           1.083
           50       50       00:00:04        0.001         0.17457
Training stopped: Max epochs completed
```




```
modelNames = ["NARX";"HW";"NSS";"SSARX";"LSTM"];
evapMeanR2 = zeros(2,5);
evapPredictions = cell(1,2);
for r = 1:2
    Yvalid = validData{r}.OutputData;
    validInput = iddata([],validData{r}.InputData,Ts);
    narxRun = sim(narxModels{r},validInput,resetOptions);
    predictions = {narxRun.OutputData,hwPredictionsEvap{r}, ...
        nssPredictionsEvap{r},ssarxPredictions{r},lstmPredictionsEvap{r}};
    evapPredictions{r} = predictions;
    totalVariation = sum((Yvalid-mean(Yvalid,1)).^2,1);
    for a = 1:5
        errors = Yvalid-predictions{a};
        r2 = 1-sum(errors.^2,1)./totalVariation;
        evapMeanR2(r,a) = mean(r2);
    end
end
smallEvapScores = table(modelNames,evapMeanR2(1,:)', ...
    VariableNames=["Model","MeanValidationR2"])
```

```
smallEvapScores = 5×2 table
```

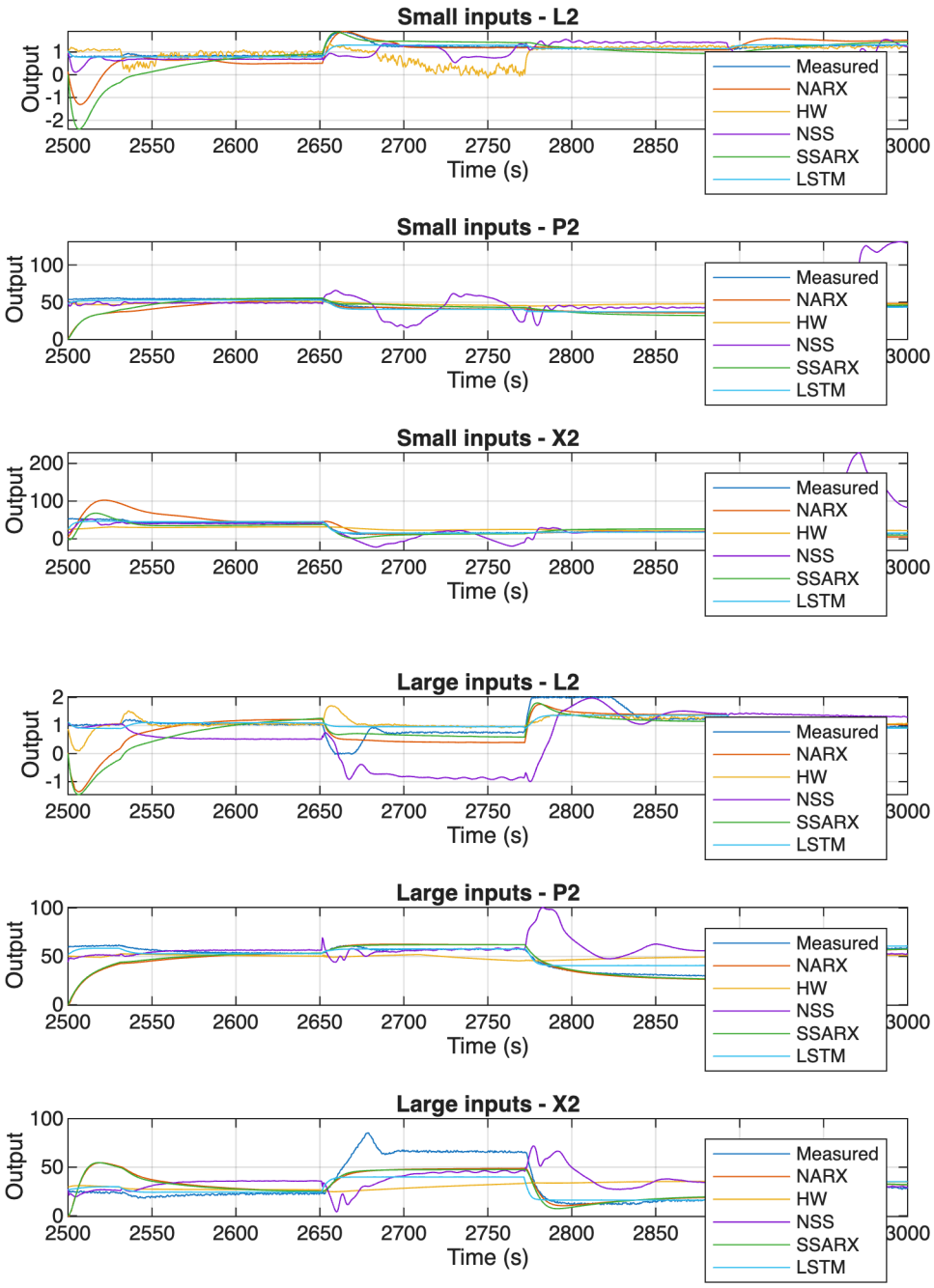
	Model	MeanValidationR2
1	"NARX"	-0.7371
2	"HW"	-0.7280
3	"NSS"	-7.9908
4	"SSARX"	-1.7735
5	"LSTM"	0.8706

```
largeEvapScores = table(modelNames,evapMeanR2(2,:)', ...
    VariableNames=["Model","MeanValidationR2"])
```

```
largeEvapScores = 5×2 table
```

	Model	MeanValidationR2
1	"NARX"	0.0879
2	"HW"	0.0479
3	"NSS"	-1.6797
4	"SSARX"	-0.0217
5	"LSTM"	0.5253

```
for r = 1:2
    Yvalid = validData{r}.OutputData;
    predictions = evapPredictions{r};
    tValid = (nTrain:nTrain+size(Yvalid,1)-1)'*Ts;
    figure;
    tiledlayout(ny,1);
    for j = 1:ny
        nexttile;
        curves = Yvalid(:,j);
        for a = 1:5
            curves = [curves,predictions{a}(:,j)];
        end
        plot(tValid,curves);
        title(regimeNames(r)+" inputs - "+string(outputNames{j}));
        xlabel("Time (s)"); ylabel("Output"); grid on;
        legend(["Measured";modelNames]);
    end
end
```



LSTM has the highest R2 score with the only one having a positive score. Given that the Evaporator model's outputs (liquid level, pressure, concentration) are based on it's inputs (flow rates, pressure) further proves that LSTM is the correct model to use here.

We noticed that performance changed with input amplitude, but the effect depended on the model. Models like NARX, HW, NSS, and SSARX has a higher R2 scores with large-amplitude inputs, while LSTM performed better with small inputs. Between the non-linear models, only LSTM performed well while the others were outperformed by linear models like HW.

Reflection and Academic Integrity Declaration

Approach Summary

For the first two parts, I generated small- and large-amplitude APRBS inputs and simulated the assigned system to obtain its outputs. I split the data into training and validation sets, then trained LSTM and Neural State-Space (NSS) models.

The main MATLAB toolkits and functions used were:

- **Control System Toolbox:** `lsim` to generate the assigned system's response.
- **Deep Learning Toolbox:** `lstmLayer`, `trainingOptions`, and `trainnet` to create and train LSTM networks, and `predict` to obtain their outputs.
- **System Identification Toolbox:** `idNeuralStateSpace`, `createMLPNetwork`, and `nlssest` to create and train NSS models, with `sim` and `evaluate` for simulation and prediction.

I compared simulation performance, one-step and five-step predictions, and clean versus noisy datasets. MSE, NMSE, R^2, and residual correlation plots were used to assess the results.

For the evaporator model, I used the supplied small- and large-input datasets, with the first 5,000 samples for training and the remaining samples for validation. I tested NARX orders using `nlarx`, trained a Hammerstein–Wiener model using `nlhw`, and estimated a state-space model using `n4sid` with the SSARX option. I also trained new NSS and LSTM models for the evaporator. The models were compared using validation simulation scores and plots of measured versus simulated outputs.

Key Insight

A nonlinear system does not automatically require the most complex model to obtain the best results. Although NSS and LSTM can represent nonlinear relationships, their performance still depends on the data, model structure, and training settings.

Another important observation was that good one-step predictions or decreasing training loss do not guarantee good simulation results. A model can predict accurately when given past measured outputs but accumulate large errors when it must continue using its own outputs. This made consistent evaluation methods and validation data important when comparing models.

Most Difficult Part

The evaporator model was the most difficult part because the different model structures produced inconsistent results across the small- and large-input datasets. For NSS and LSTM, I had to experiment with training settings such as network size, epochs, and, for NSS, training-window size. Improving training loss did not always improve validation performance, so I also needed to inspect the simulated outputs rather than rely only on the training charts.

Disclosure on the use of AI Systems

- **Plot Enhancements** - Codex - I used this mostly for assisting in generating the charts and tables to visually show the results.
- **Research and Reference Lookup** - Codex - I used Codex for helping easily search all our notes as well as MATLAB documentation to speed up research time and focus more on implementation.